

第三章 中断和处理器调度

提纲

3.1 中断与中断系统

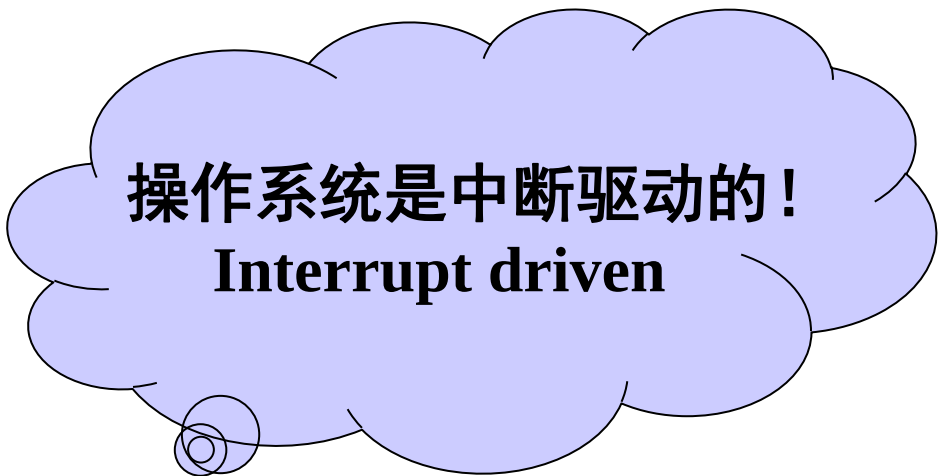
3.2 处理器调度

3.3 调度级别与多级调度

3.4 实时调度

3.5 多处理器调度

3.6 系统举例



操作系统是中断驱动的！
Interrupt driven

提纲

3.1 中断与中断系统

3.2 处理器调度

3.3 调度级别与多级调度

3.4 实时调度

3.5 多处理器调度

3.5 系统举例

3.1 中断与中断系统

- 3.1.1 中断的概念
- 3.1.2 中断装置
- 3.1.3 中断处理程序

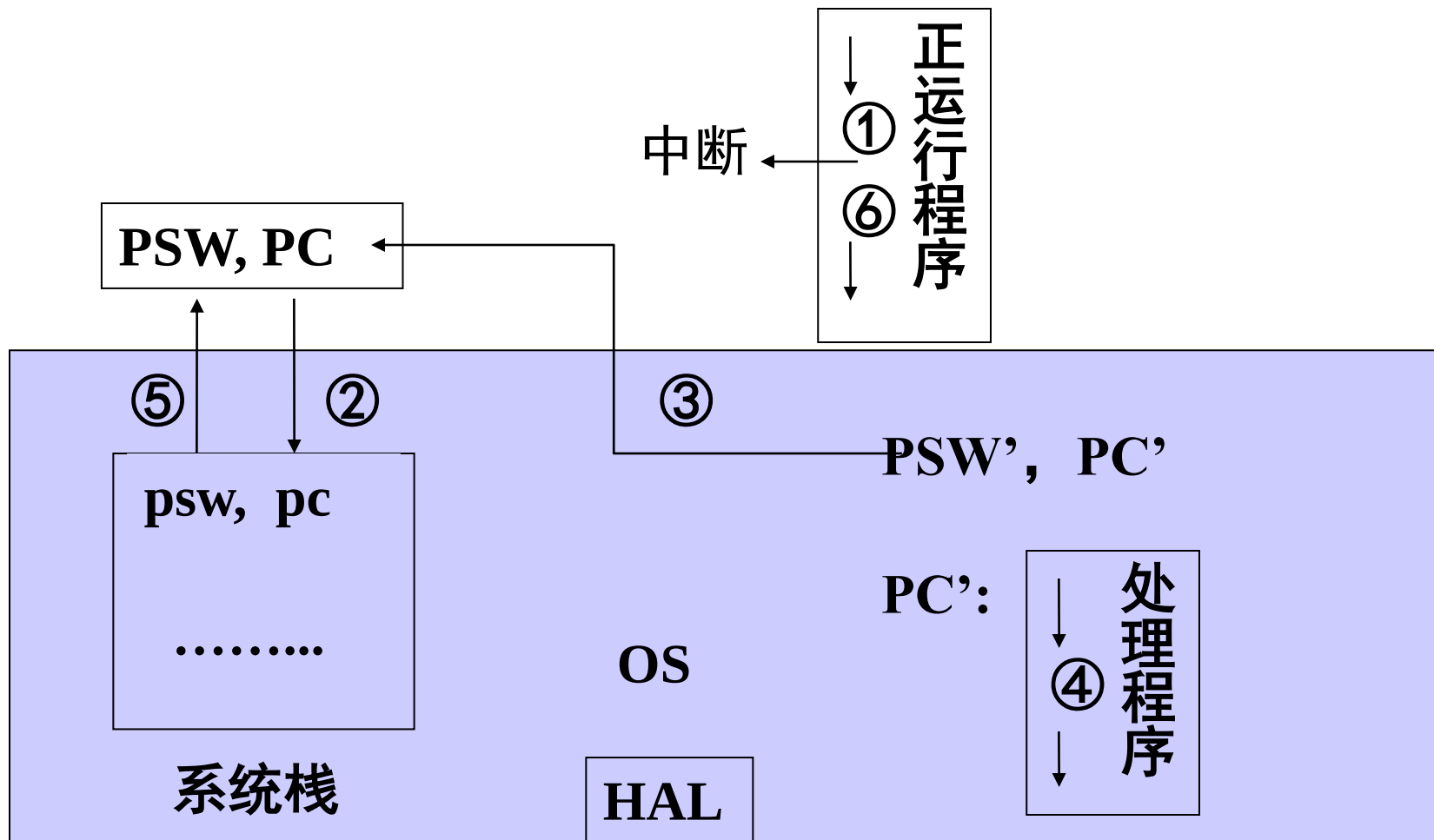
3.1.1 中断的概念

- 中断
 - 在程序运行过程中出现某紧急事件，必须中止当前正在运行的程序，转去处理这个事件，然后再返回原来运行的程序，这一过程称为中断。
- 中断系统
 - 中断装置(硬件)
 - 中断处理程序(软件)

3.1.2 中断装置

- 发现并响应中断的硬件机构
 - 识别中断源，当有多个中断源时，按紧迫程度排队
 - 保存现场
 - 引出中断处理程序

中断响应和处理的過程



3.1.2.1 中断源与中断字

- 中断源
 - 引起中断的事件
- 中断寄存器
 - 保存与中断事件相关信息的寄存器
 - 有些中断不需要中断寄存器，如溢出
- 中断字
 - 中断寄存器的内容
- 例：I/O中断
 - 中断源：I/O
 - 中断寄存器：保存着与设备相关的信息
 - 中断字：内容

3.1.2.2 中断类型与中断向量

- 中断类型
 - 强迫性中断
 - 自愿性中断
- 中断类型
 - 外部中断
 - 由CPU外部事件引起，指来源于CPU之外的中断事件
 - 例：I/O，时钟中断，外部信号的中断
 - 内部中断
 - 由CPU内部事件引起
 - 例：访管中断、程序中断

3.1.2.2 中断类型与中断向量

■ 强迫性中断

■ 运行程序不期望的

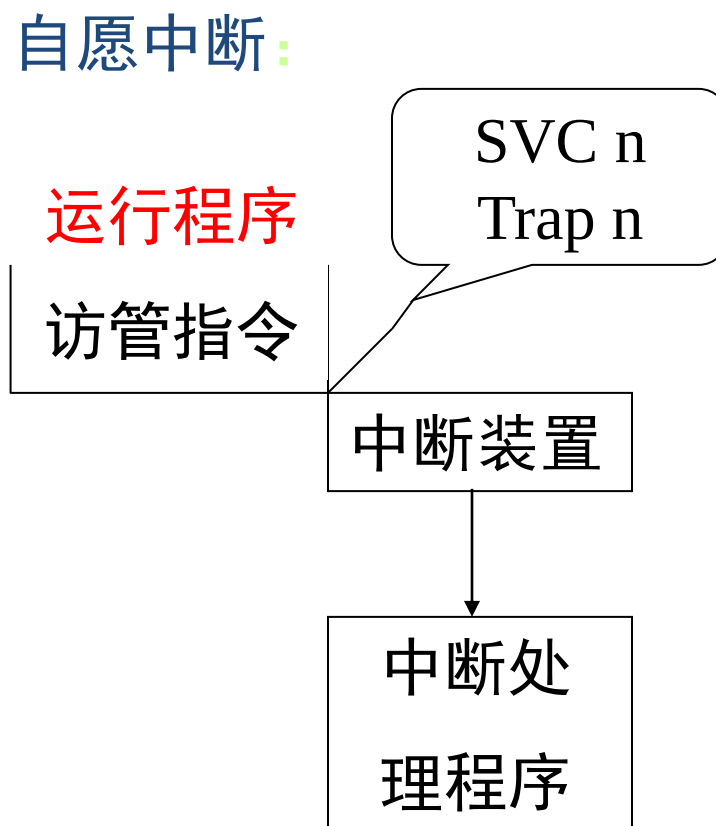
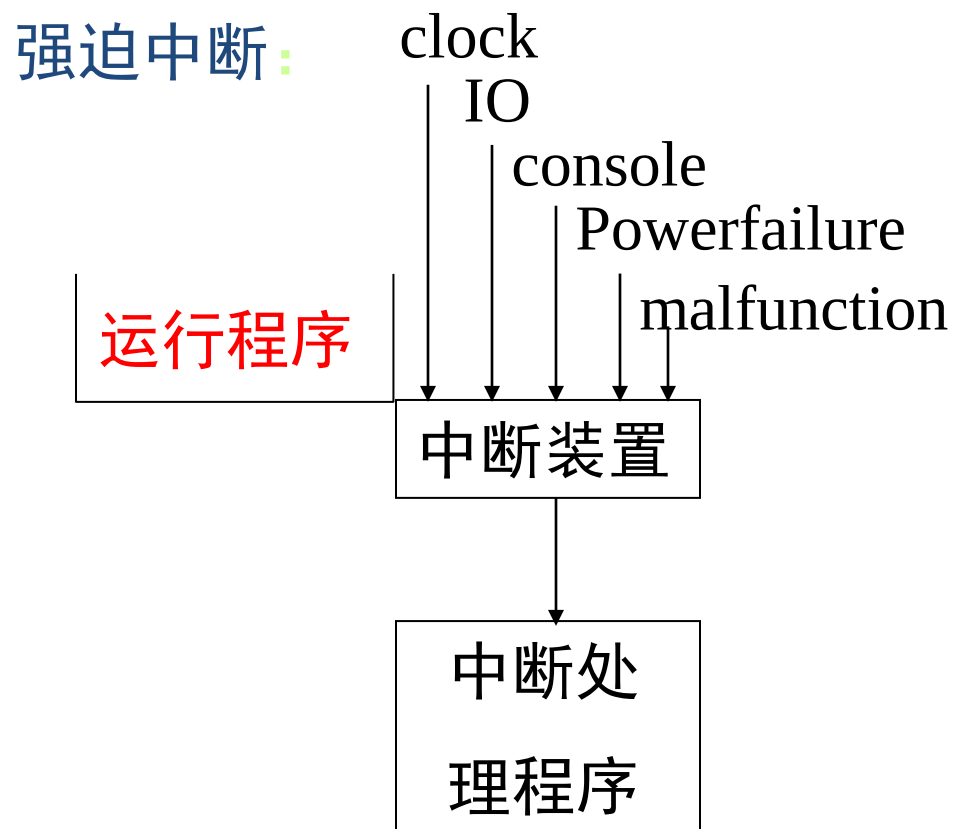
- 时钟中断
- I/O中断
- 控制台中断
- 键盘, 鼠标中断
- 硬件故障中断
 - power failure
 - 内存校验错误
- 程序性中断
 - 越界, 越权
 - 缺页, 缺段
 - 溢出, 除0
 - 非法指令

■ 自愿性中断

■ 运行程序期望的

- 系统调用
 - 访管指令
- 系统调用
 - `fd=open(fname, mode)`
- 访管指令
 - 准备参数
 - `svc n`
 - 取返回值

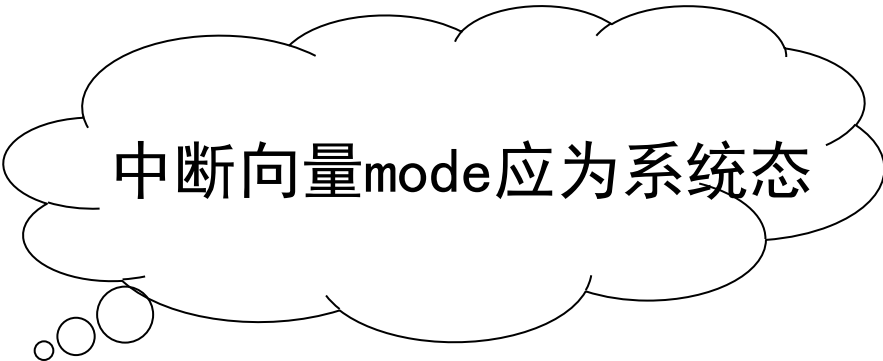
3.1.2.2 中断类型与中断向量(Cont.)



3.1.2.2 中断类型与中断向量(Cont.)

■ 中断向量

- 中断处理程序的运行环境与入口地址 (PSW, PC)
- 每类中断事件有一个中断向量
- 中断向量的存放位置是由硬件规定的
- 中断向量的内容是OS在系统初始化时设置好的.



中断向量mode应为系统态

3.1.2.2 中断类型与中断向量(Cont.)

0000	PSW1, PC1	时钟中断向量
0008	PSW2, PC2	I/O中断向量
0016	PSW3, PC3	console中断向量
0024	PSW4, PC4	硬件故障
0030	PSW5, PC5	程序错误

...

...

...

0090	PSW _n , PC _n	访管中断向量
------	------------------------------------	--------

PC1: 时钟中断
处理程序

PC2: I/O中断
处理程序

...

PC_n: 访管中断
处理程序

系统空间

3.1.2.3 中断嵌套与系统栈

- 中断嵌套
 - 系统在处理一个中断事件的过程中又响应了新的中断
- 中断优先级
 - 硬件规定的中断响应次序，依据
 - 紧迫程度
 - 处理时间

3.1.2.3 中断嵌套与系统栈

- 一般原则
 - 高优先级别中断可以嵌入低优先级中断
- 实现方法
 - 中断响应后立即屏蔽不低于当前中断优先级的中断源
- 中断屏蔽
 - 高优先级中断事件处理不受低优先级中断打扰
 - 硬件系统提供中断屏蔽指令,
 - 程序调整中断响应次序

3.1.2.3 中断嵌套与系统栈(Cont.)

中断响应后一般需要进一步保存现场

关中断（屏蔽所有中断）

进一步保存现场（通用寄存器等）

开中断（或开放高优先级中断）

...

中断处理

...

关中断（屏蔽所有中断）

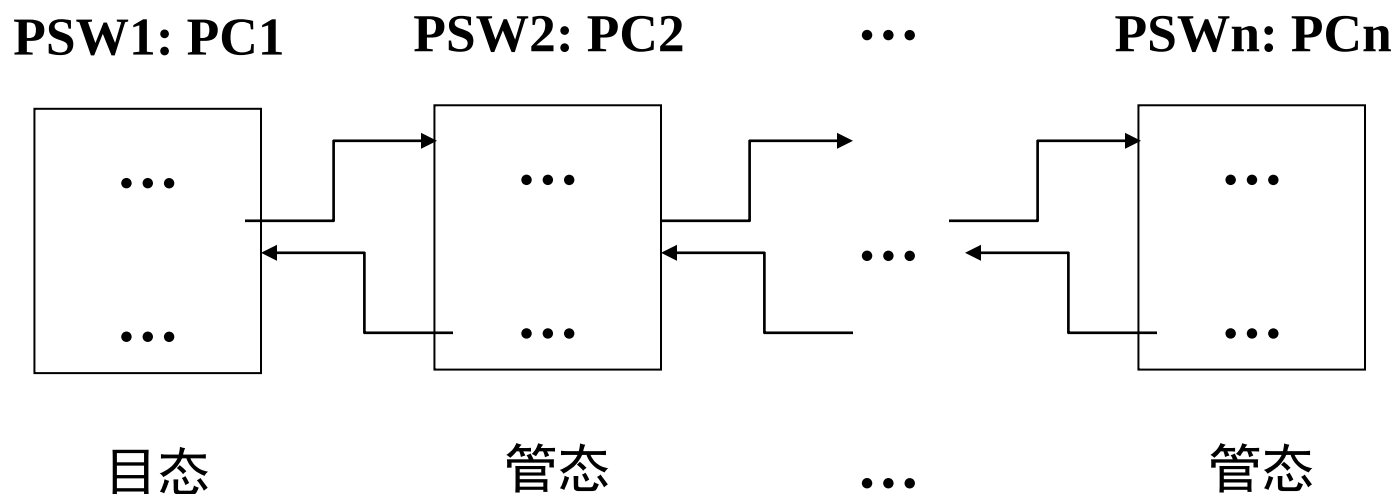
恢复现场

开中断

中断返回

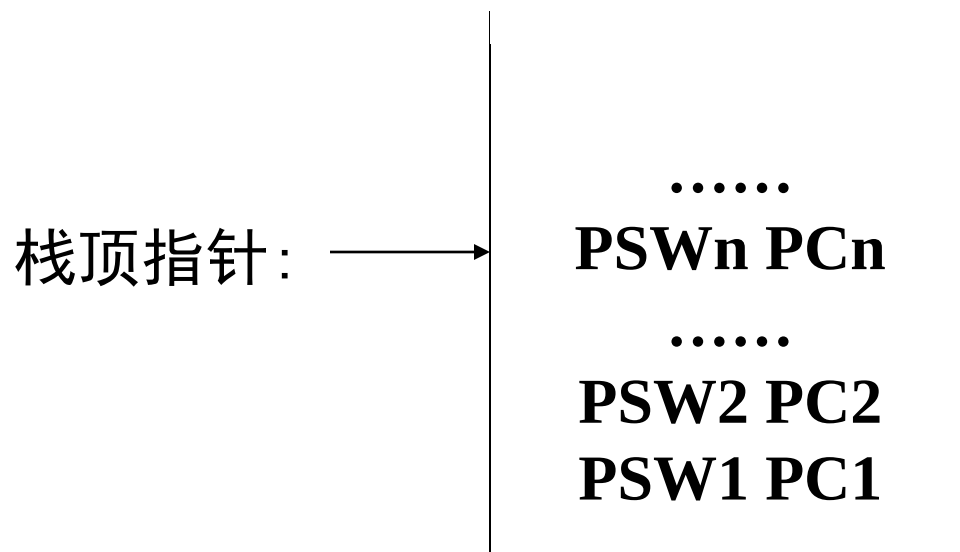
3.1.2.3 中断嵌套与系统栈(Cont.)

中断嵌套:



3.1.2.3 中断嵌套与系统栈(Cont.)

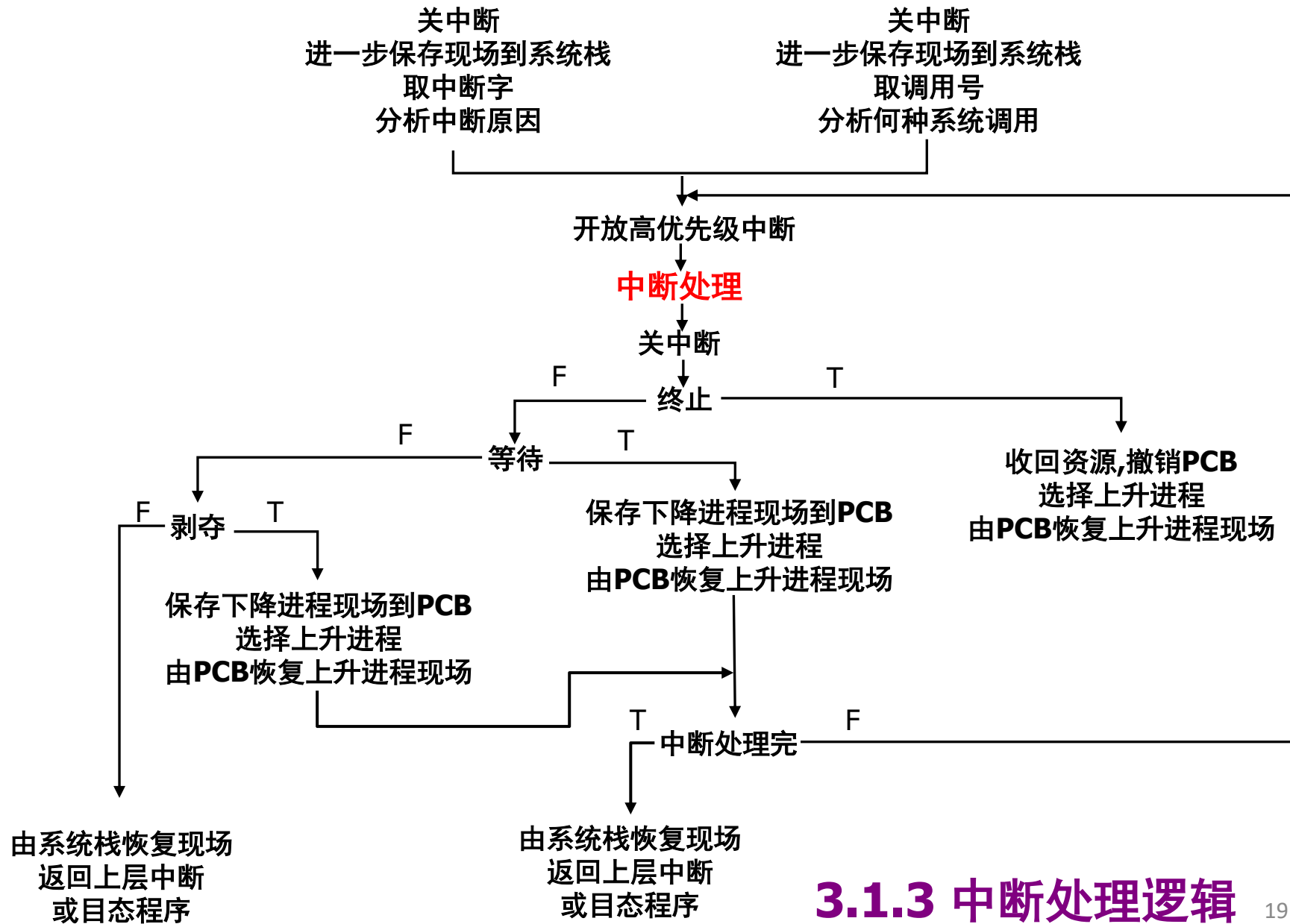
系统栈：



中断相当于一种特殊的子程序调用，发生时刻具有不确定性

强迫性中断:

自愿性中断:



3.1.3 中断处理逻辑

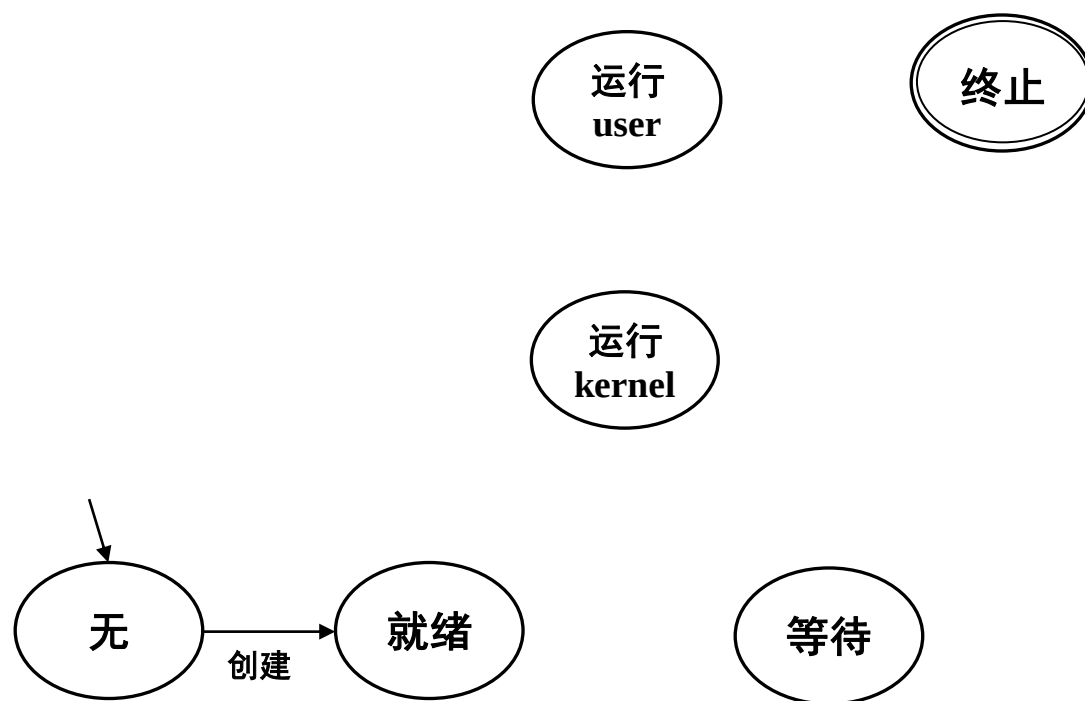
中断分析

- 关于等待
 - 何时等待？
 - 处于核心态, 无嵌套中断或有嵌套
 - 等待几次？
 - 可能多次
 - 保存在PCB的是什么级别现场？
 - 核心级别
 - 等待时系统栈如何？
 - 栈底是目态现场, 然后是嵌套函数的返回点, 参数, 局部变量, 返回值; 如有嵌套, 接下来是核心现场, 然后是嵌套函数的返回点、参数、局部变量、返回值; (可能有多重)

中断分析

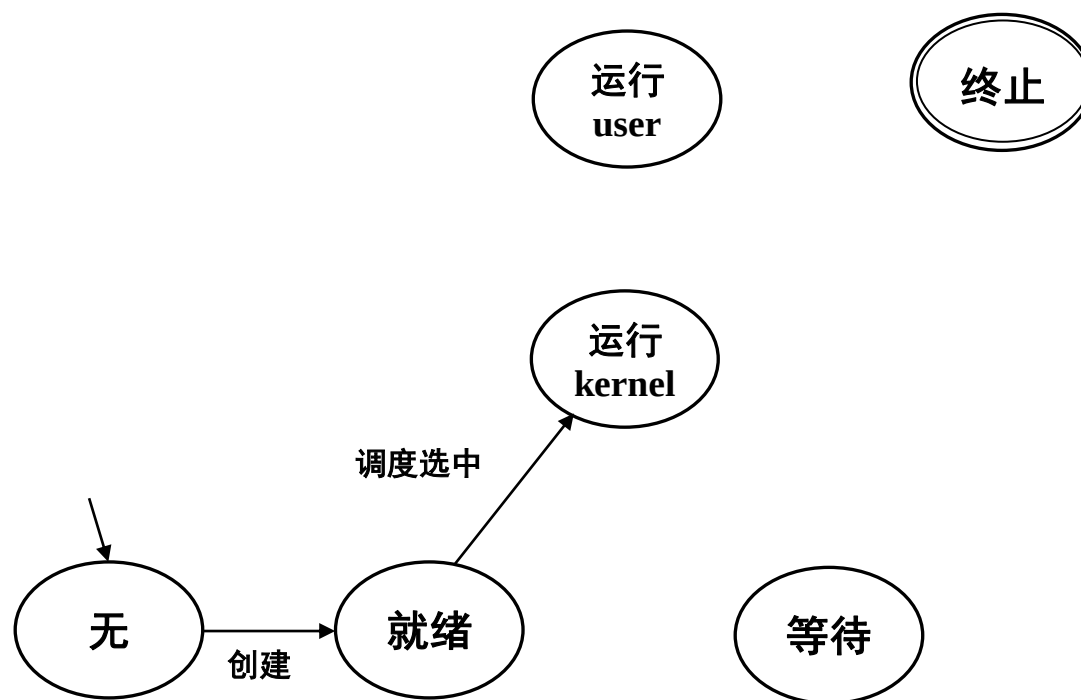
- 关于剥夺
 - 何时等待剥夺？
 - 处于核心态, 无嵌套中断或有嵌套
 - 等待剥夺几次？
 - 可能多次
 - 保存在PCB的是什么级别现场？
 - 核心级别
 - 剥夺时系统栈如何？
 - 栈底是目态现场, 然后是嵌套函数的返回点, 参数, 局部变量, 返回值; 如有嵌套, 接下来是核心现场, 然后是嵌套函数的返回点、参数、局部变量、返回值; (可能有多重)

考虑系统状态的进程状态转换图



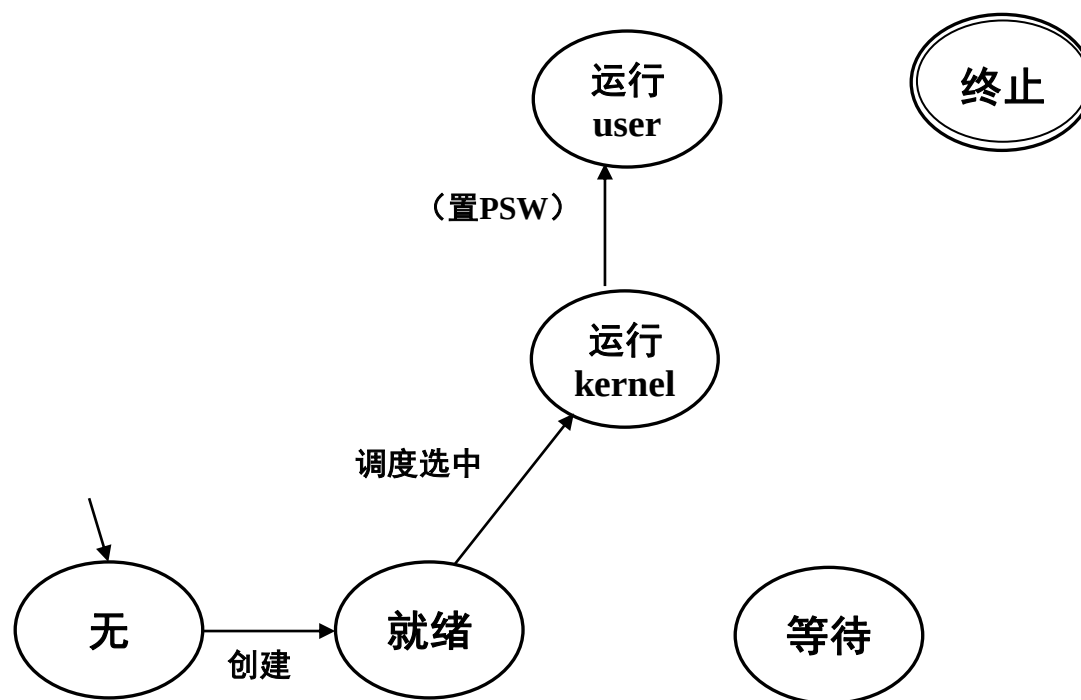
建立PCB, 分配资源, 加载程序, 初始化PCB (地址映射寄存器, PSW (目态), PC=0, SP=栈底, regs=0, fregs=0), 入就绪队列

考虑系统状态的进程状态转换图



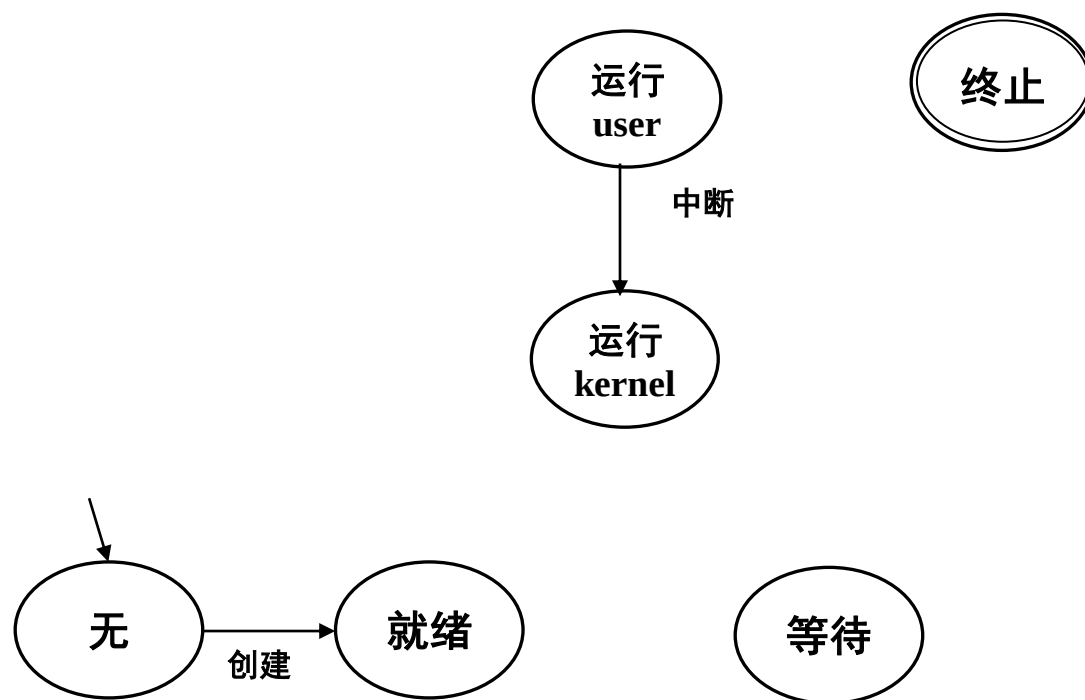
PCB切换，恢复地址映射寄存器，通用(浮点)寄存器，SP

考虑系统状态的进程状态转换图



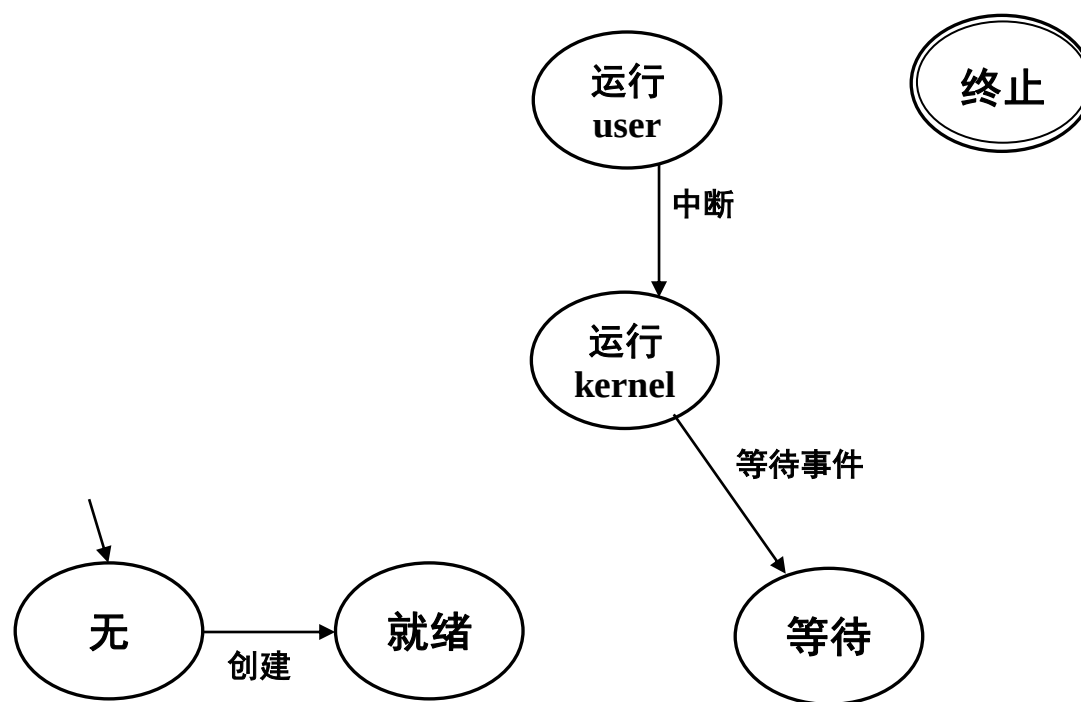
恢复PSW和PC

考虑系统状态的进程状态转换图



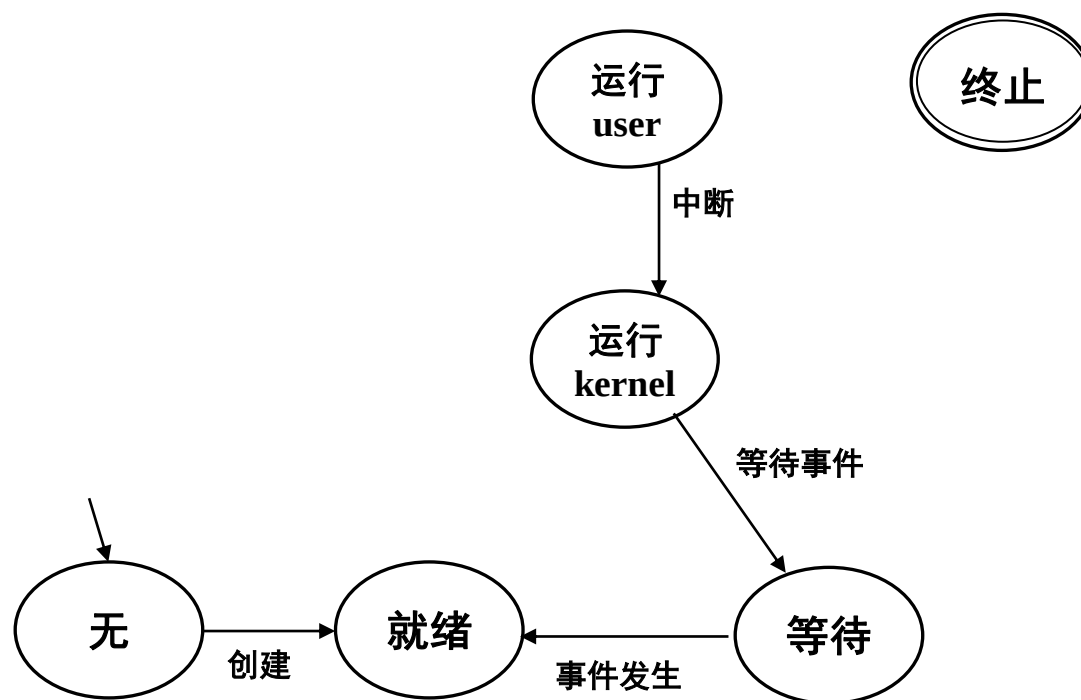
现场在系统栈

考虑系统状态的进程状态转换图



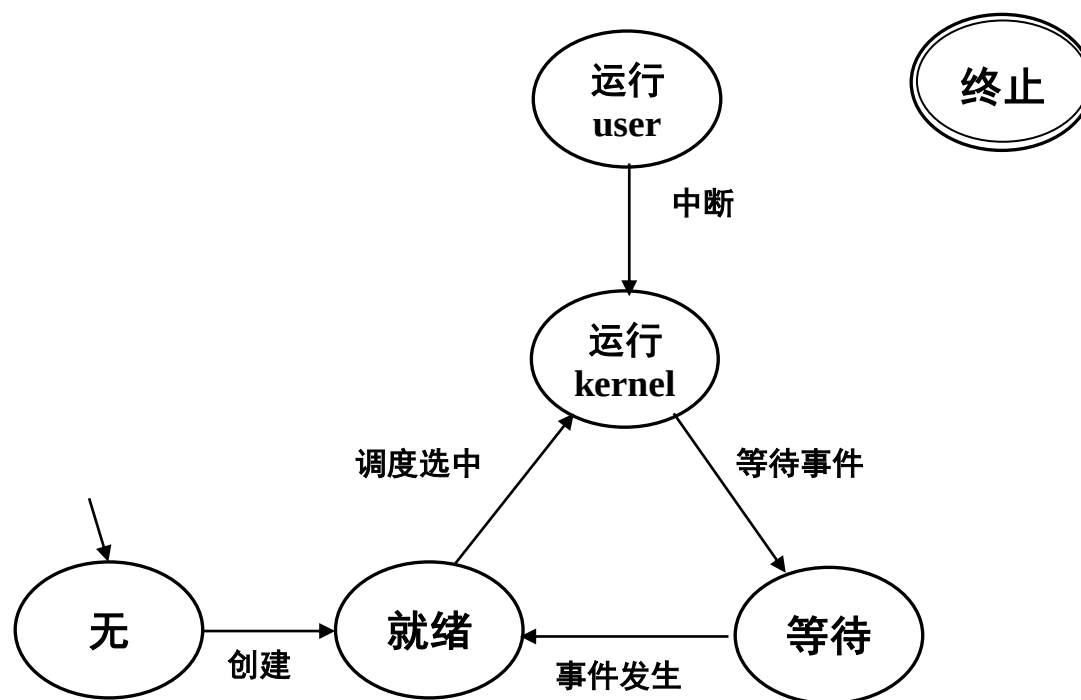
保存核心级别现场到PCB(地址映射寄存器, regs, fregs, SP, PSW, PC), 将PCB入等待队列

考虑系统状态的进程状态转换图



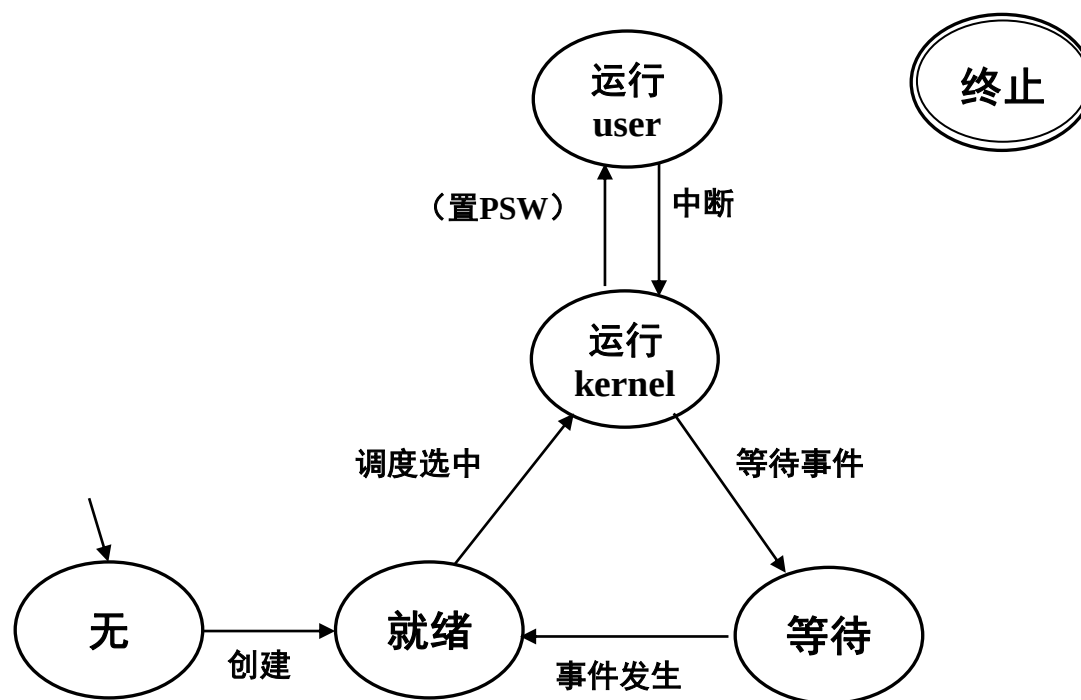
I/O中断，资源释放并分给该进程，PCB由等待队列转移到就绪队列

考虑系统状态的进程状态转换图



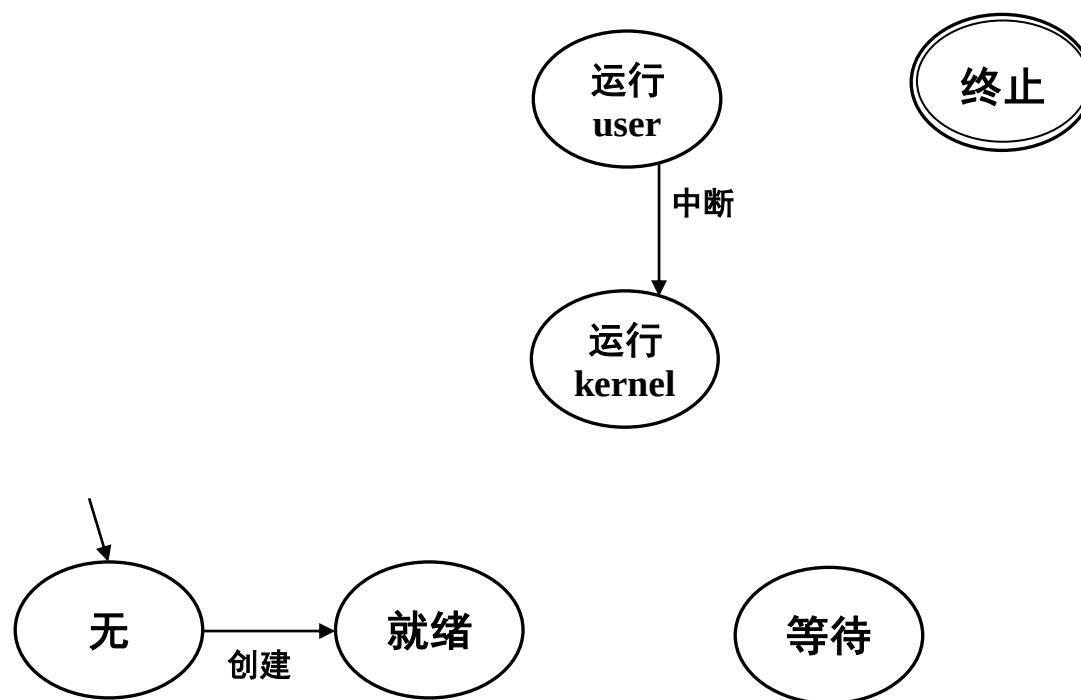
恢复现场(地址映射寄存器, regs, fregs, SP, 用户打开文件)

考虑系统状态的进程状态转换图



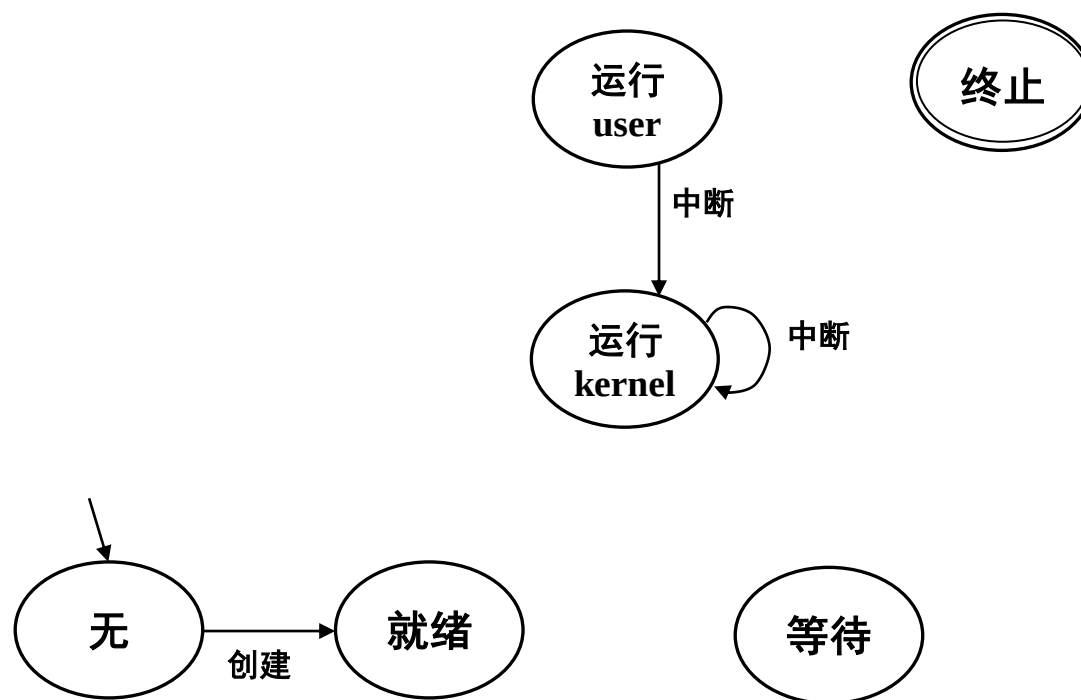
最后恢复PSW和PC，进程处于用户态下的运行

考虑系统状态的进程状态转换图



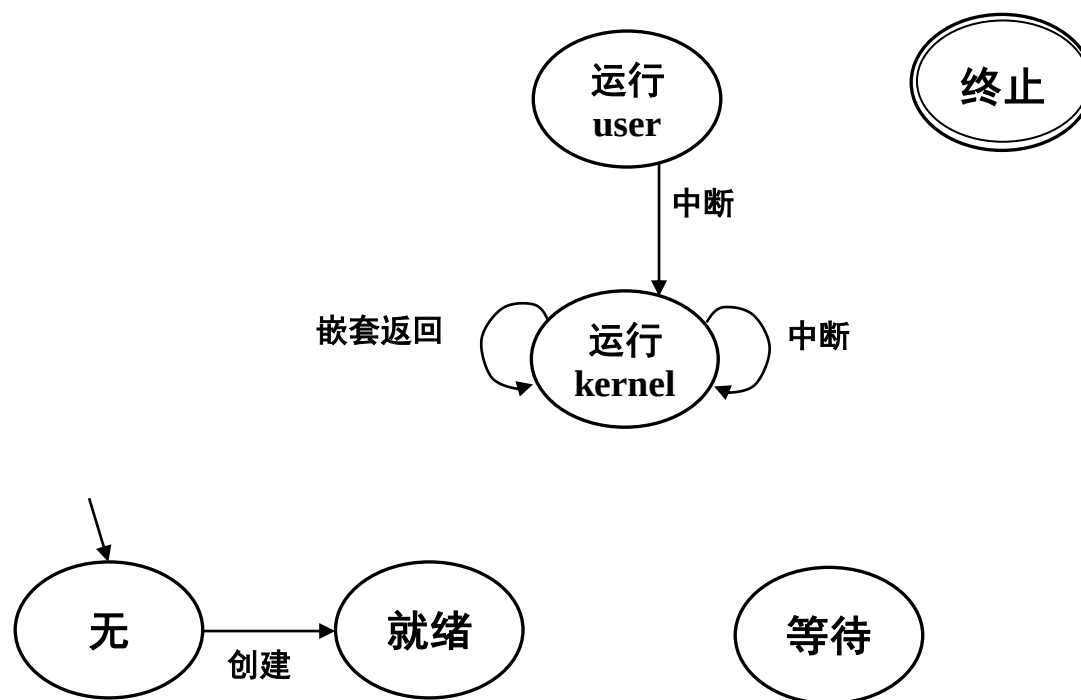
现场压入系统栈

考虑系统状态的进程状态转换图



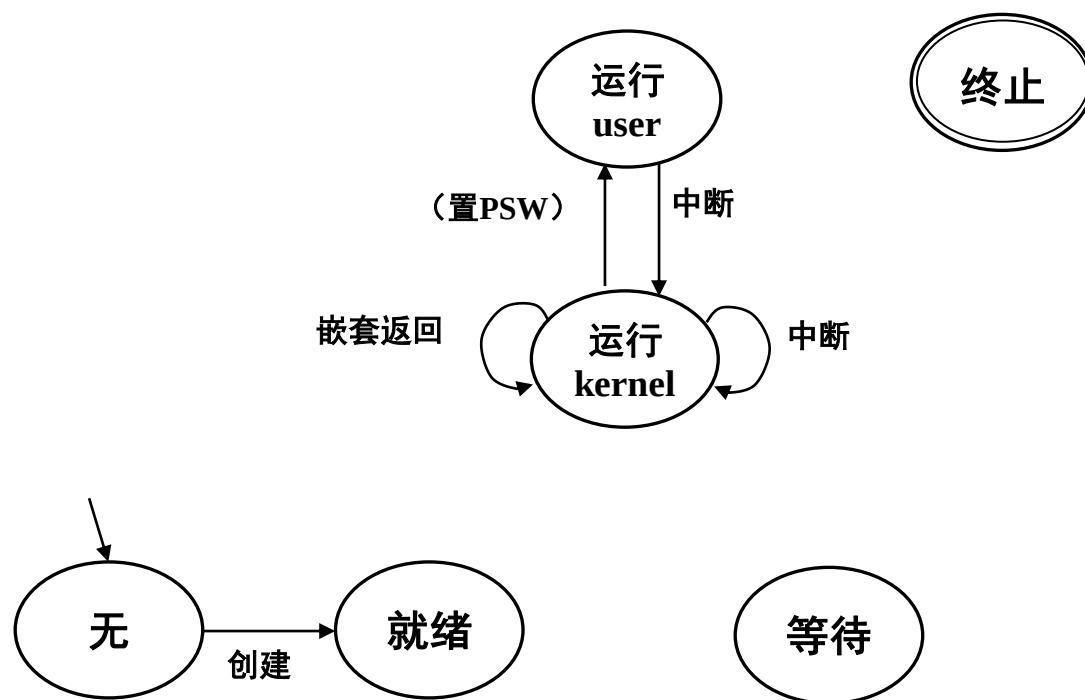
嵌套中断，现场压入系统栈

考虑系统状态的进程状态转换图



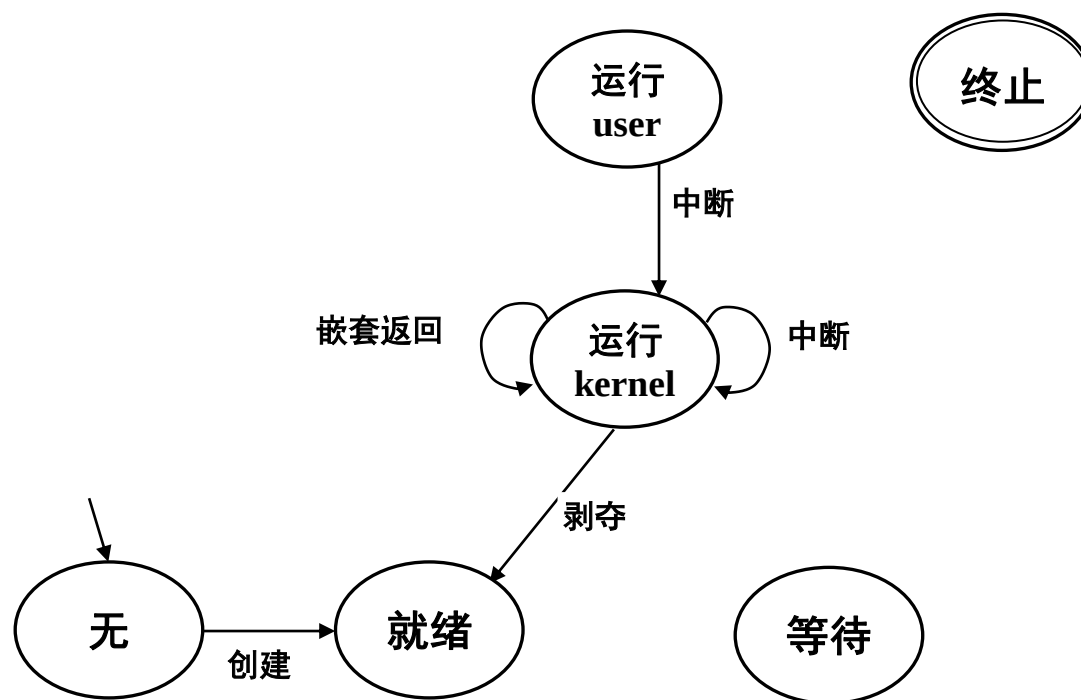
由系统栈弹出现场

考虑系统状态的进程状态转换图



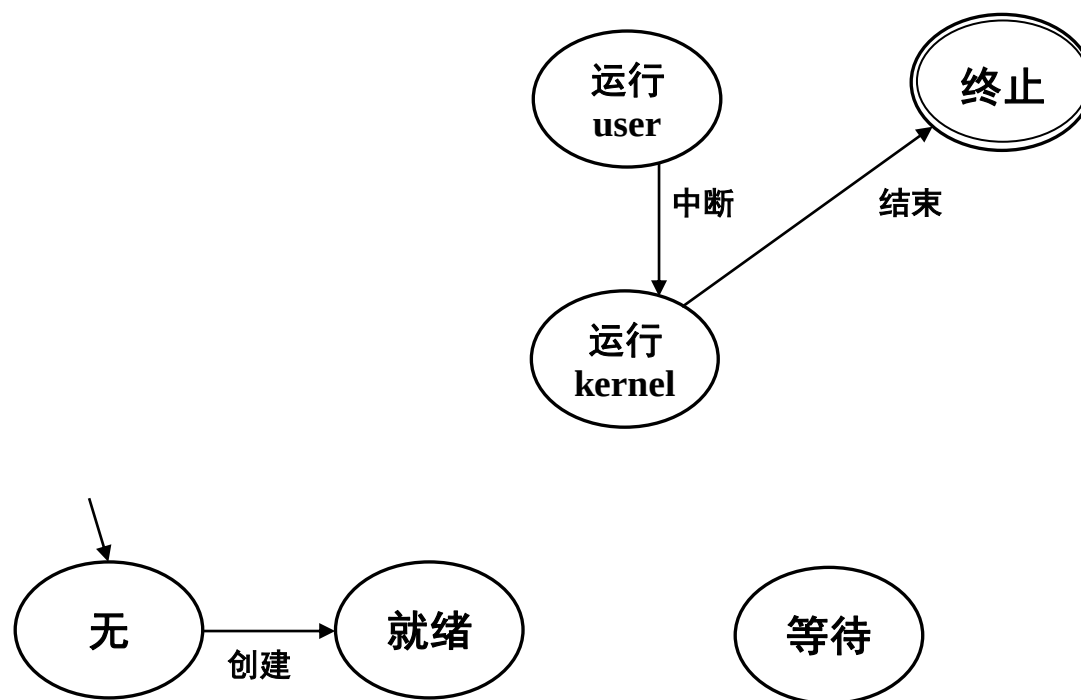
退到第一层中断，不剥夺处理器，由系统栈弹出现场

考虑系统状态的进程状态转换图



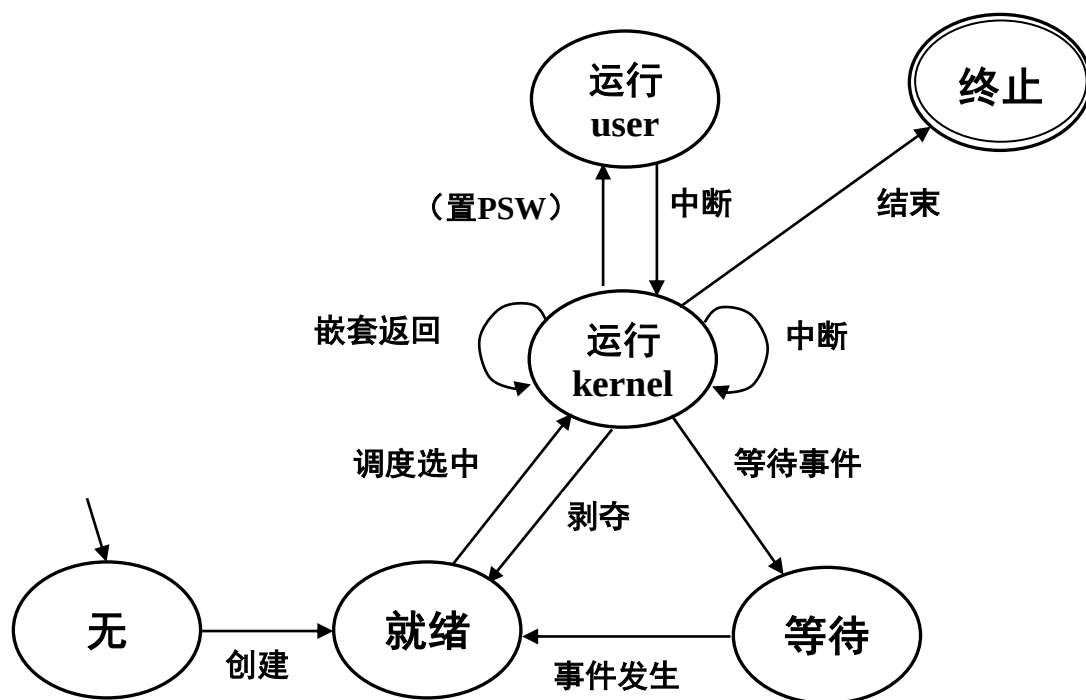
剥夺处理机，保存核心级别现场到PCB(地址映射寄存器，regs, fregs, SP, PSW, PC)，PCB入就绪队列

考虑系统状态的进程状态转换图



执行系统调用`exit()`, 收回资源, 撤销PCB

考虑系统状态的进程状态转换图



-
- 思考1：中断现场保存在什么地方？进程现场保存在什么地方？
 - 思考2：为什么说PCB保存的是核心级别现场？用户级别现场保存在什么地方？

3.1.3 中断处理程序

- I/O中断处理
- 时钟中断处理
- 控制台中断处理
- 硬件故障中断处理
- 程序性中断的处理
- 自愿性中断的处理

3.1.3.1 I/O中断处理

- I/O中断正常结束
 - 如果需要继续传输数据，准备好数据后即可启动通道
 - 如果发出I/O请求的进程正等待数据传输完成，则将其唤醒
- 数据传输错误
 - 复执（eg. 3次）
 - 若不成功，报告系统操作员

3.1.3.2 时钟中断处理

- 时钟类型
 - 硬件时钟
 - 绝对时钟：不发生中断
 - 间隔时钟：发生中断
 - 软件时钟
 - 利用硬件定时机构和程序实现，软件时钟不发生中断

3.1.3.2 时钟中断处理(Cont.)

- Housekeeping
 - 实现软时钟，启动定时程序
 - 硬时钟5ms发生一次中断，软时钟50ms
 - 进程管理
 - 重新计算进程调度参数(eg. 动态优先数)，判断是否进程切换
 - 分时系统中，记录进程已经占有处理器时间并判断时间片是否用完
 - 作业管理
 - 记录作业在输入井中等待的时间，以及当前的优先级，以便作业调度程序据此决定一个将要进入系统执行的作业
 - 资源管理
 - 动态统计运行进程占有和使用处理器等资源的时间等
 - 事件处理
 - 在实时系统中，定时向被控对象发送控制信号等
 - 系统维护，如定时运行死锁检测程序、系统记账程序

3.1.3.3 控制台中断处理

- 多个控制按钮，每个按钮对应一个中断向量
- 当按下控制台上某个按钮后，就会产生一个中断信号，指向一个中断处理程序

3.1.3.4 硬件故障中断处理

- 电源故障处理
 - 掉电：硬件设备应保证继续工作一段时间
 - 内存，寄存器⇒外存
 - 停止设备
 - 停止处理机
 - 恢复：故障排除后，可进行系统恢复
 - 启动处理机
 - 启动设备
 - 外存⇒内存，寄存器



Use UPS for critical applications

3.1.3.4 硬件故障处理(cont.)

- 内存故障处理
 - 如果一个内存单元出现错误，中断处理程序可以对错误单元进行检测并确认错误后，将其所在区域永久性划分为不可用区

3.1.3.5 程序性中断的处理

- 只能由操作系统处理的中断
 - 影响系统或其它进程
 - 越界，非法指令，（处理：终止进程）
 - 需要系统管理或协助
 - 页故障，缺段，（处理：动态调入）
- 可以由用户自己处理的中断
 - 不影响系统和其它进程
 - 除0，溢出，（处理：用户处理，或OS处理）

应用程序自己处理中断

调试语句：on <中断条件> <中断续元入口>

例如：on <divide_zero> goto LA;

除0中断时转LA处理

LA:

除0中
断续元

on <divide_zero> goto LB;

除0中断时转LB处理

LB:

除0中
断续元

相同中断发生在不同位置
可采用不同处理方法

应用程序自行处理中断 (Cont.)

编译时：为每个用户程序生成一个中断续元表（用户态）

中断事件0:	中断续元入口0	} 初始时均为0
中断事件1:	中断续元入口1	
...		
中断事件n:	中断续元入口n	

运行时：执行调试语句，填写中断续元表

中断时：根据中断原因查中断续元表

为0，用户未规定中断续元，由OS标准处理

非0，用户已规定中断续元，由用户处理

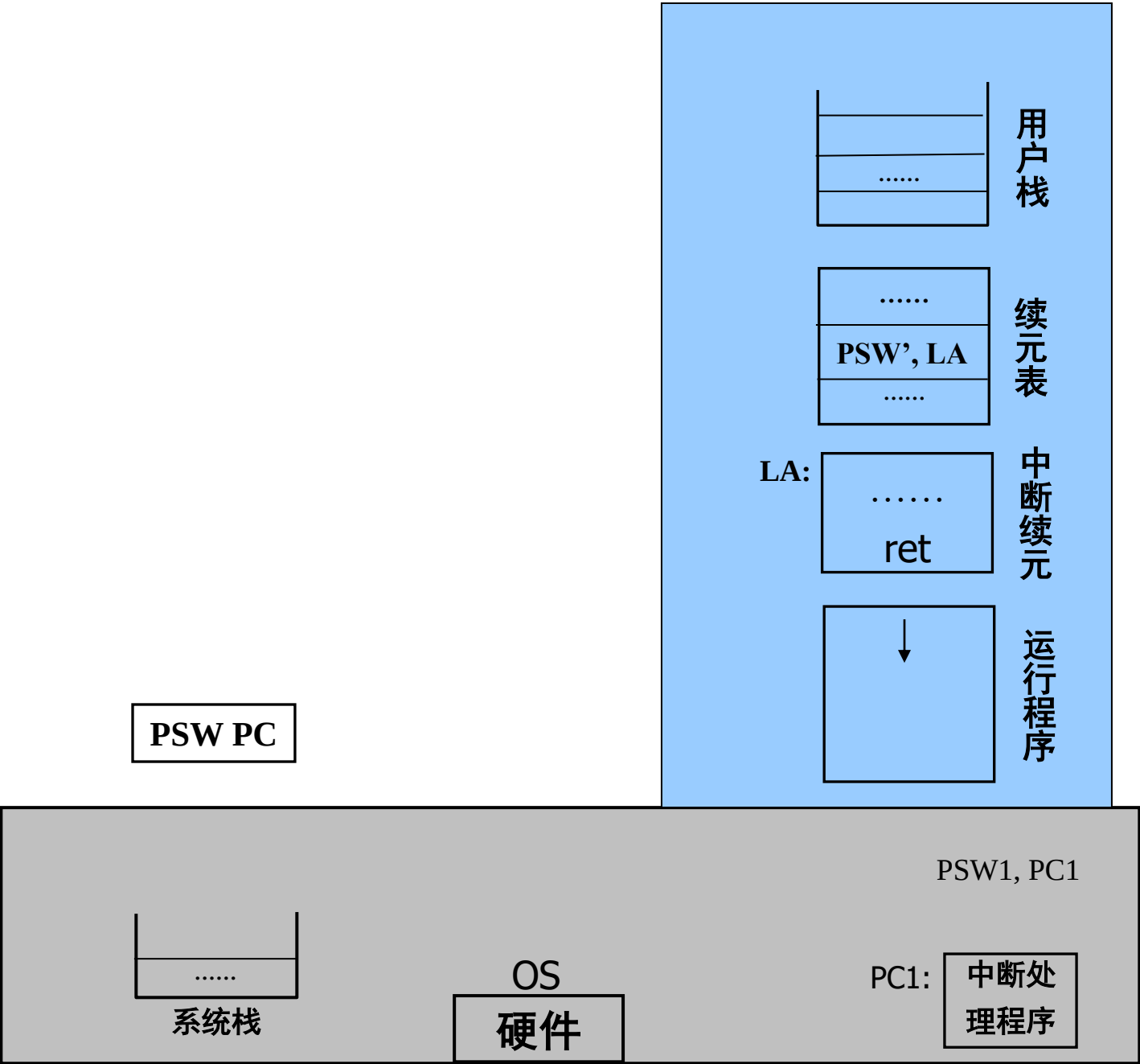
应用程序自行处理中断 (Cont.)

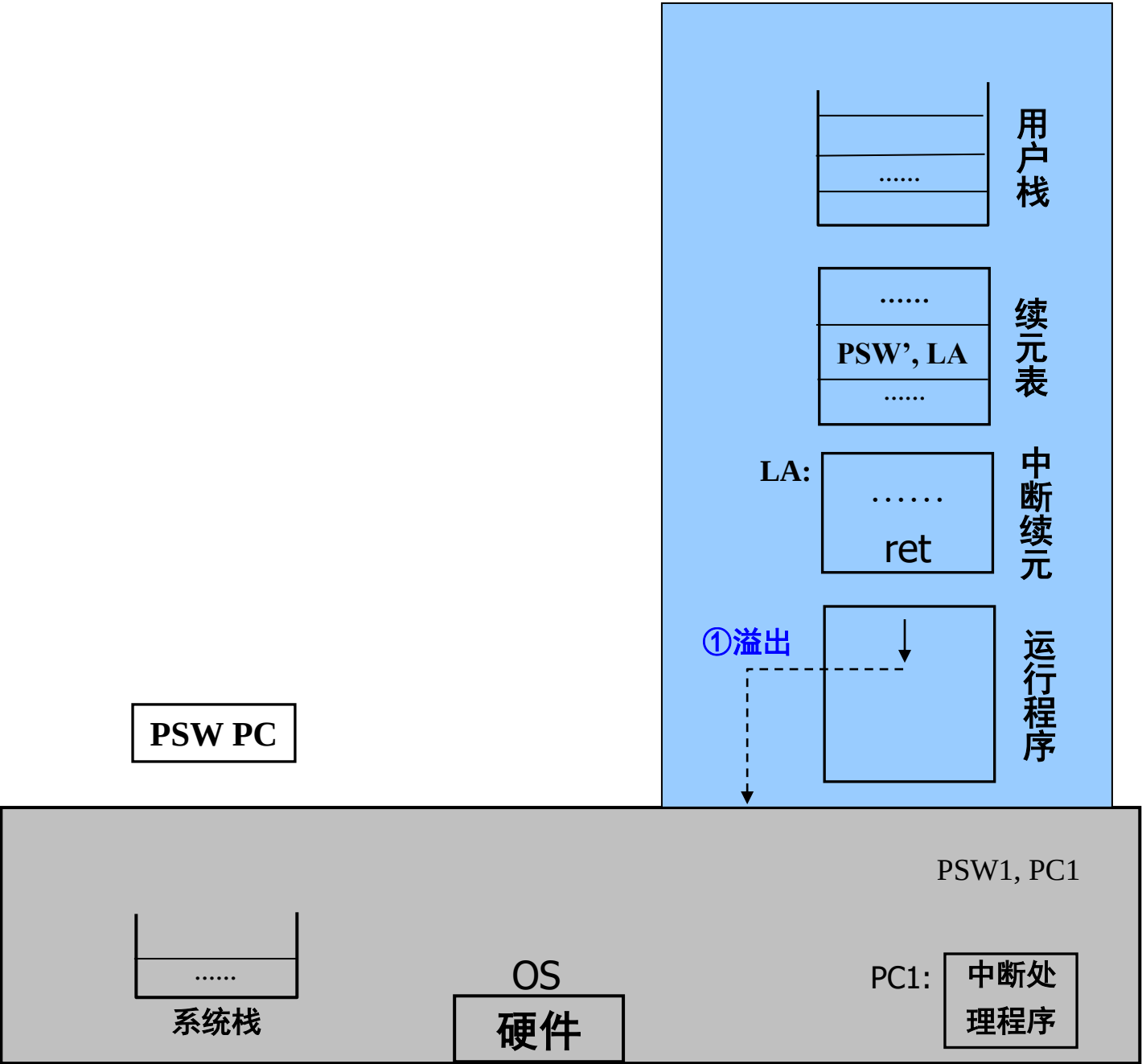
- 如何转到中断续元？
- 如何返回被中断的用户程序？

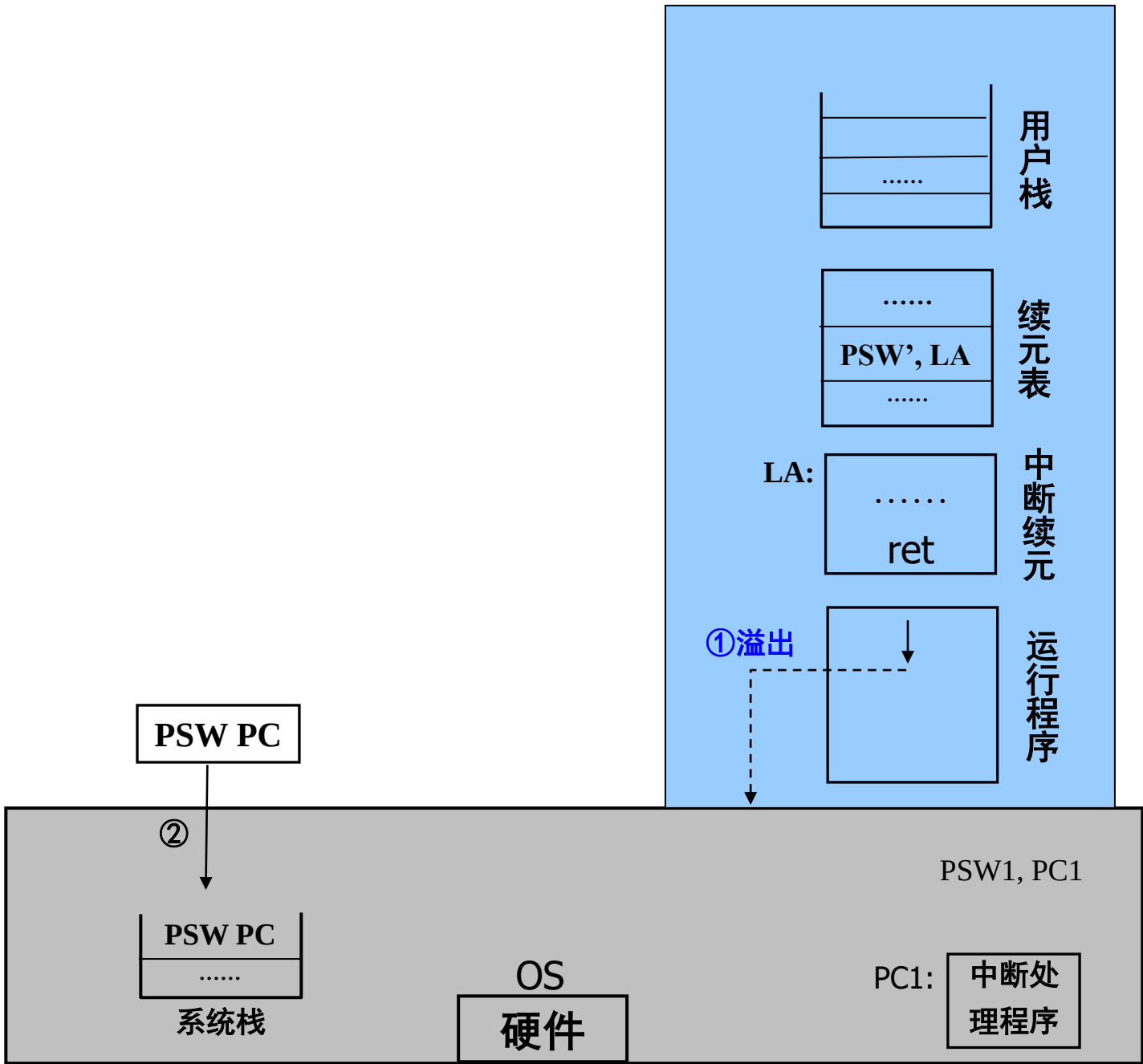
应用程序自行处理中断(Cont.)

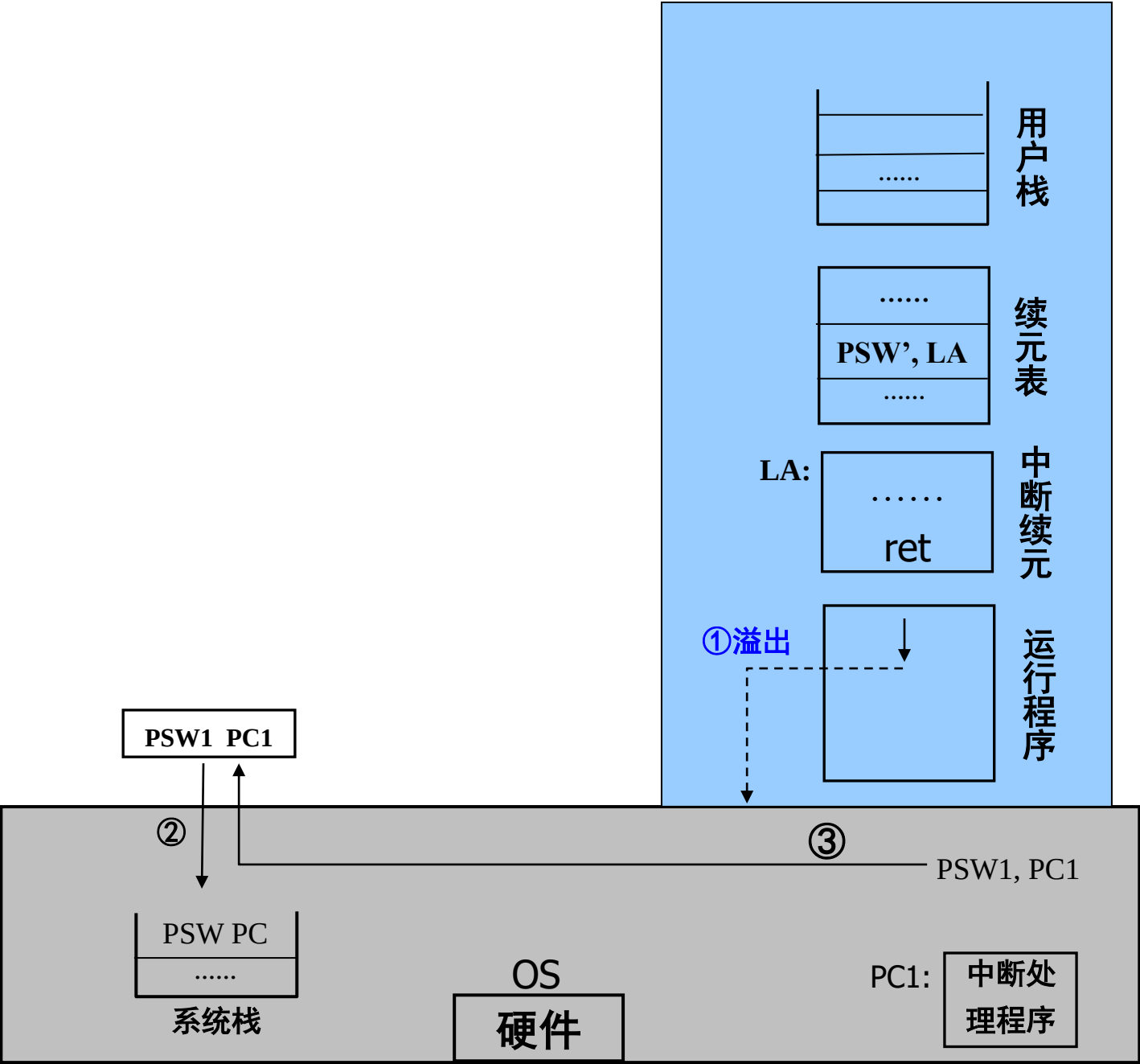
■ 步骤:

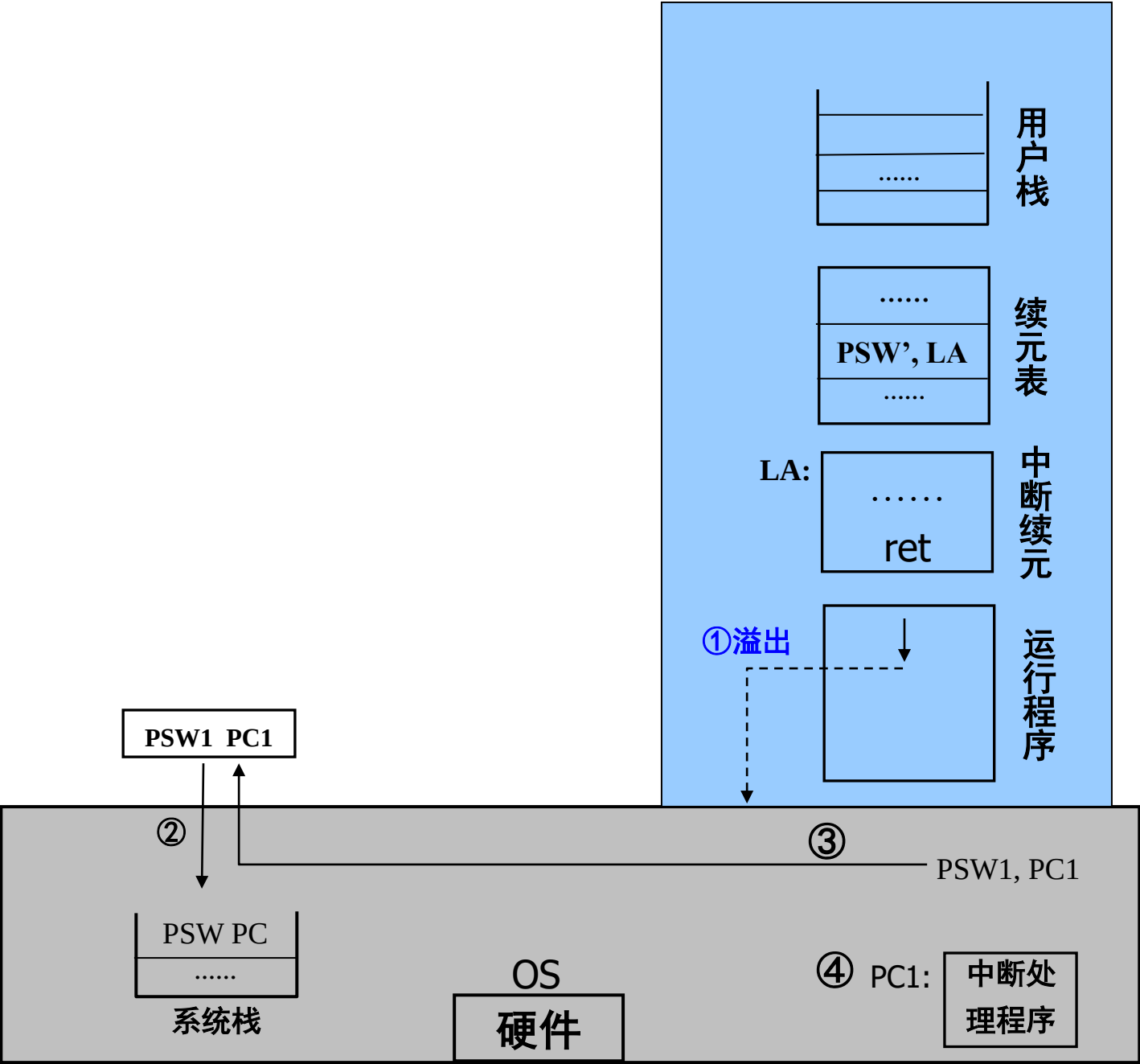
- (1) 目态程序在运行时发生溢出中断
- (2) 中断装置将PSW和PC值压入系统栈
- (3) 取中断向量并送入PSW和PC寄存器
- (4) 执行对应的操作系统中断处理程序
- (5) 访问用户程序中的中断续元表（假定非0）
- (6) 系统栈中现场转移到用户栈
- (7) 中断续元运行环境和入口送PSW和PC
- (8) 控制权转到中断续元，执行中断续元
- (9) 由用户栈弹出现场送PSW和PC
- (10) 返回断点继续执行

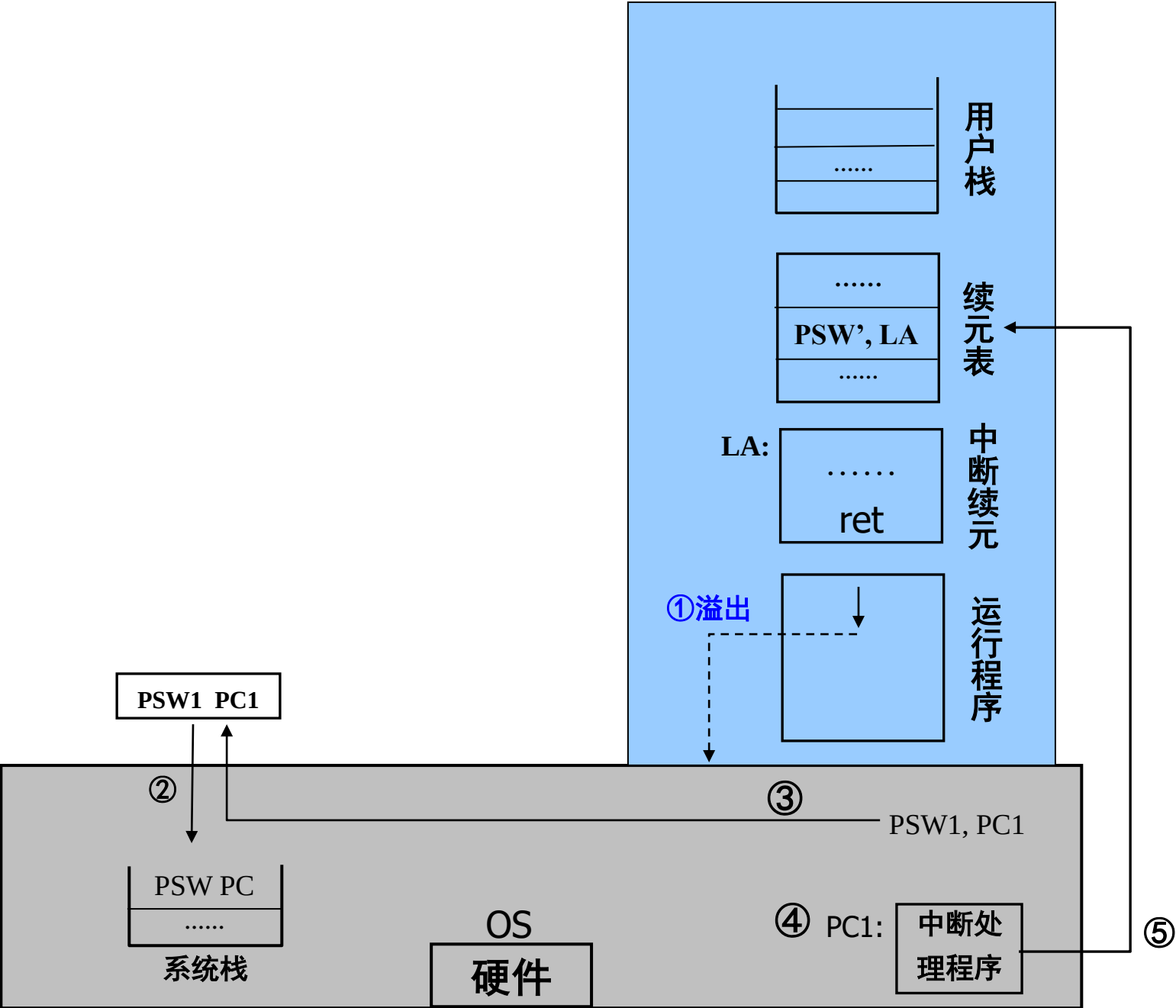


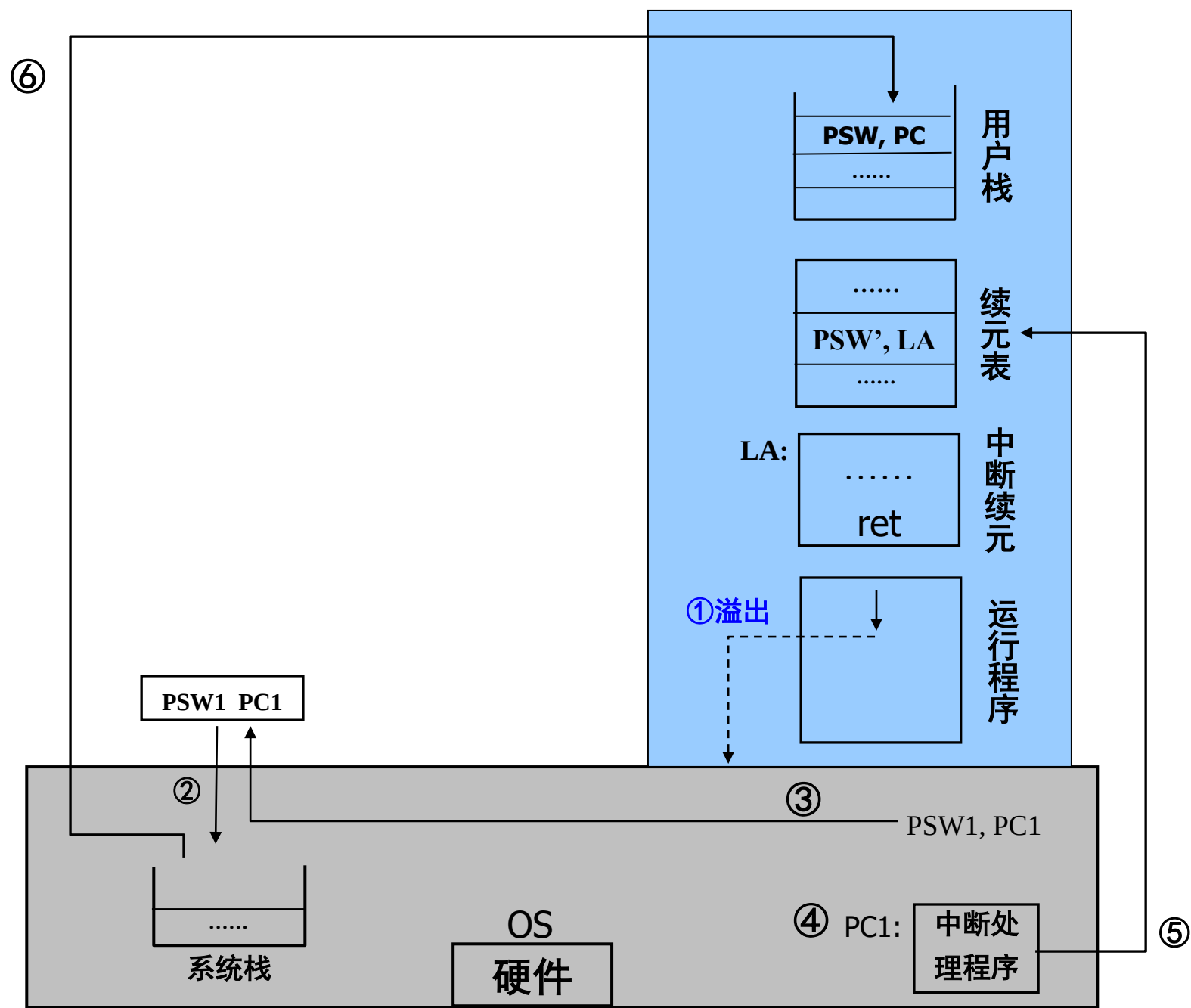


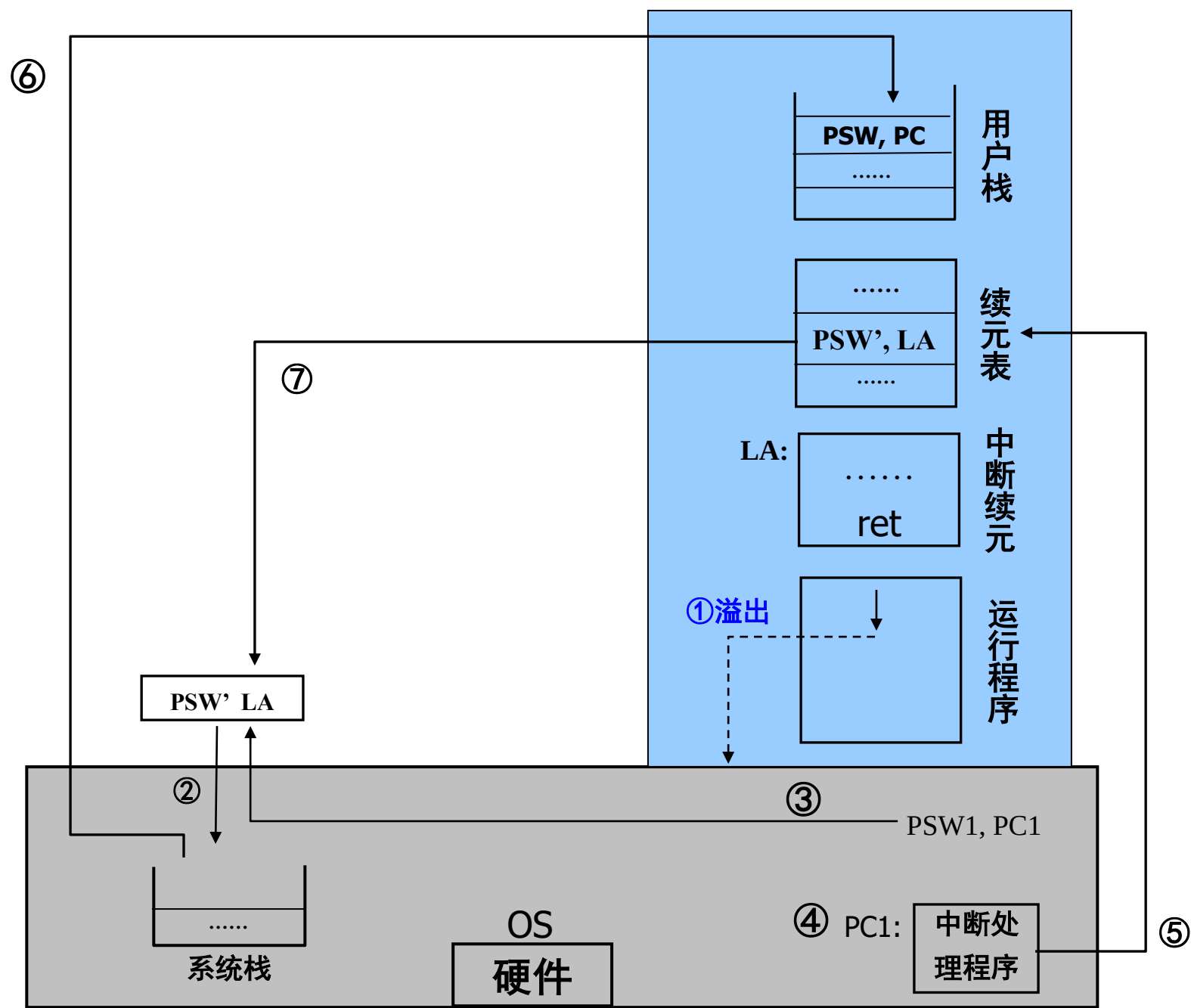


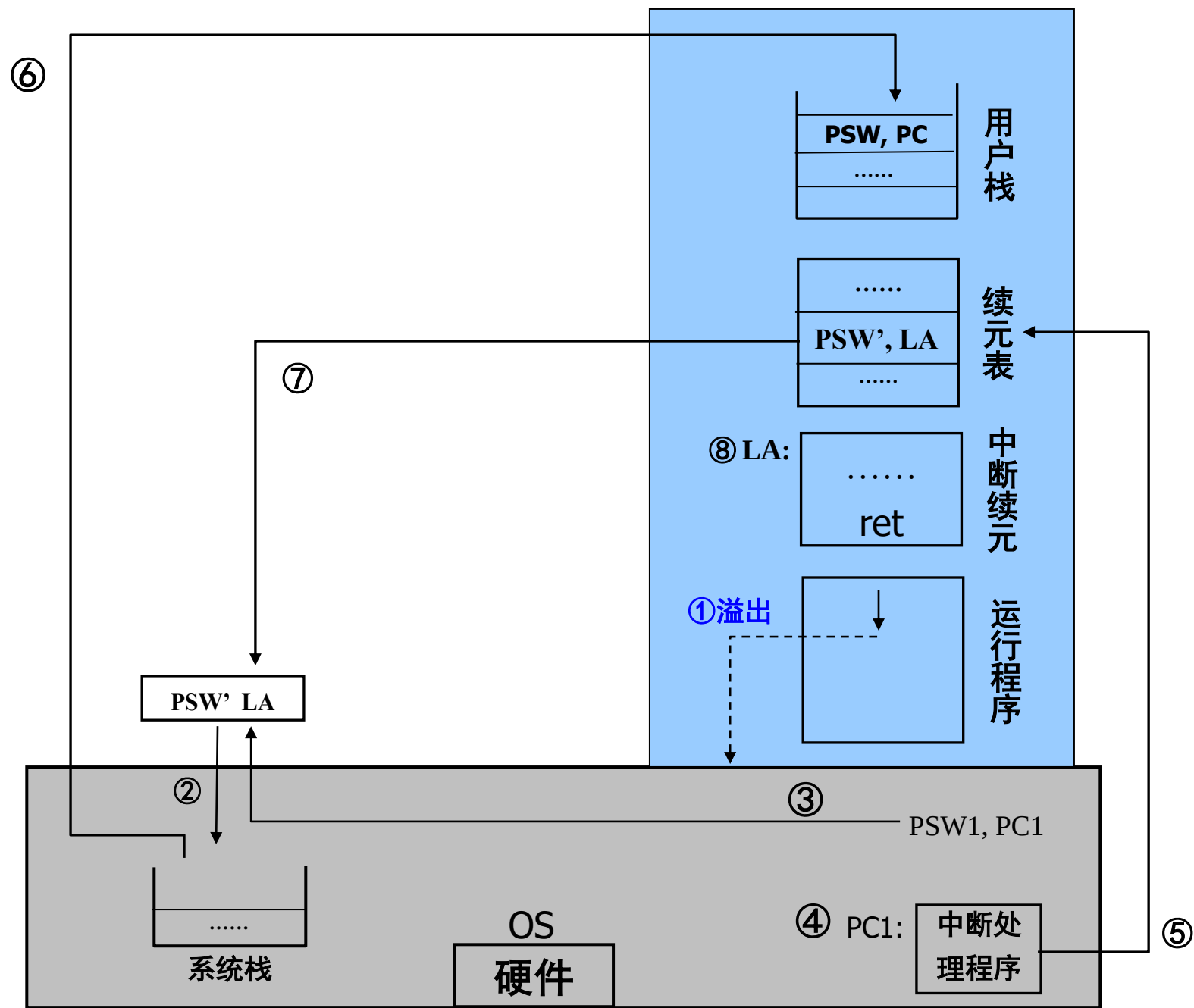


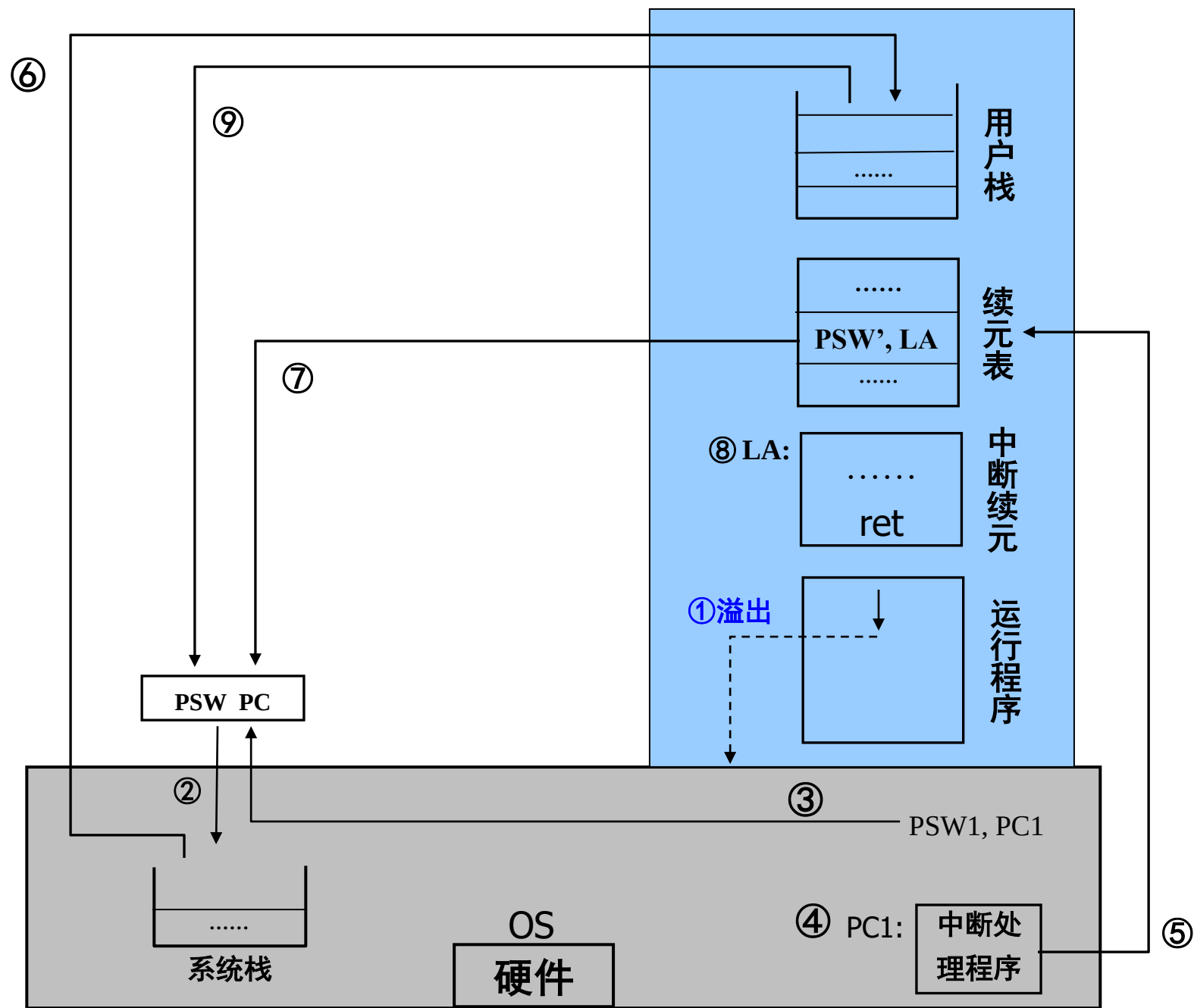


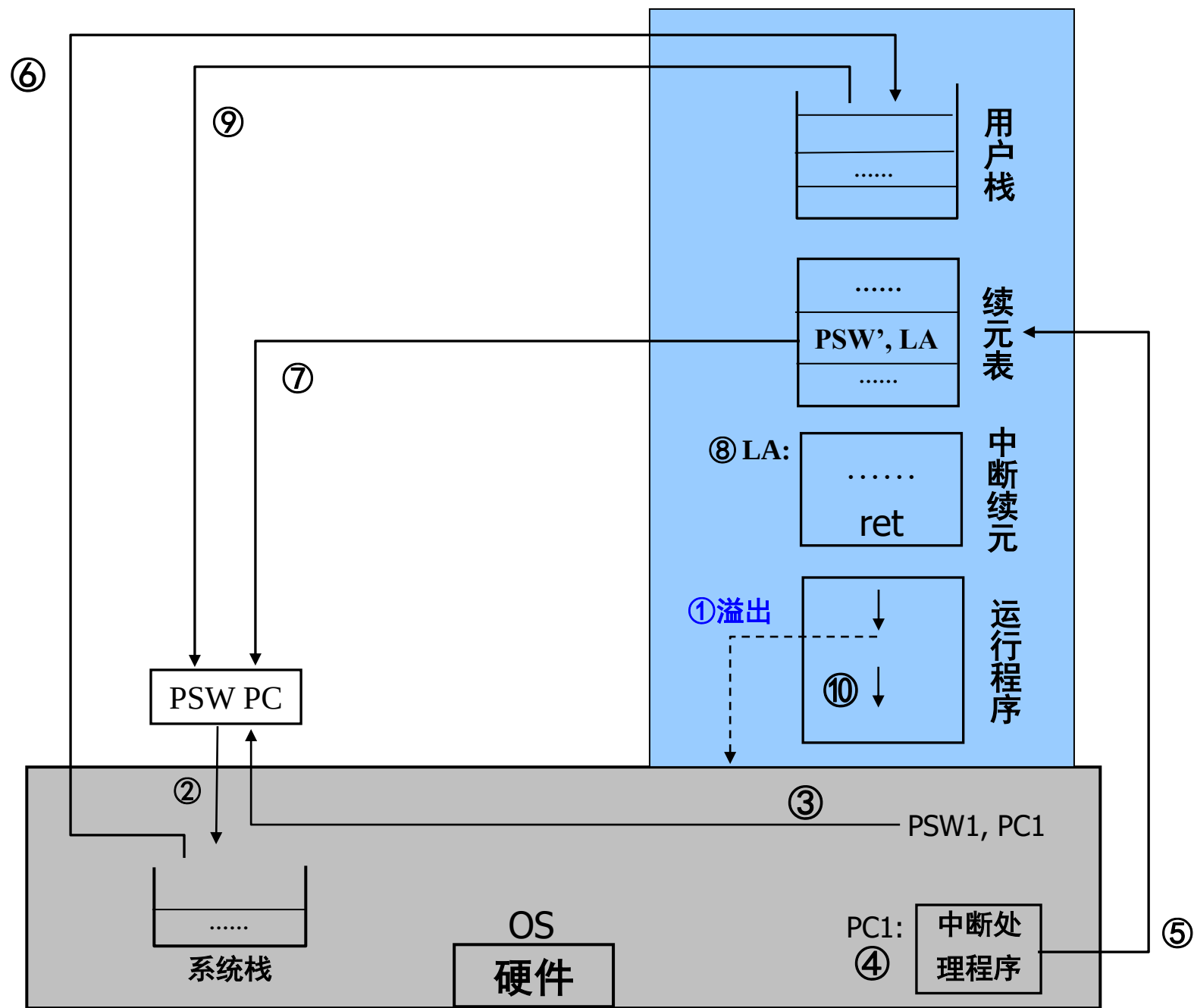












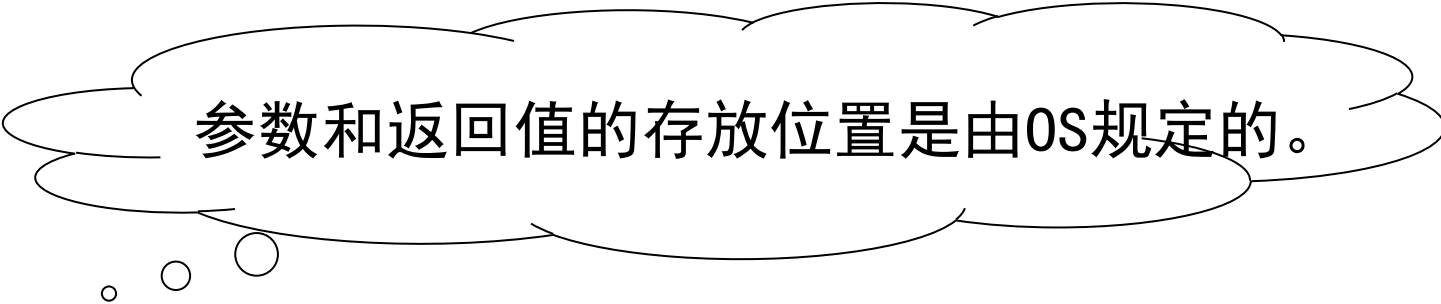
3.1.3.6 自愿性中断的处理

访管指令 (SuperVisor Call) 形式:

准备参数

SVC n

取返回值



参数和返回值的存放位置是由OS规定的。

系统调用 (system call) 形式:

返回值=系统调用名称 (实参1, ..., 实参n)

编译程序会将系统调用形式翻译为访管指令形式

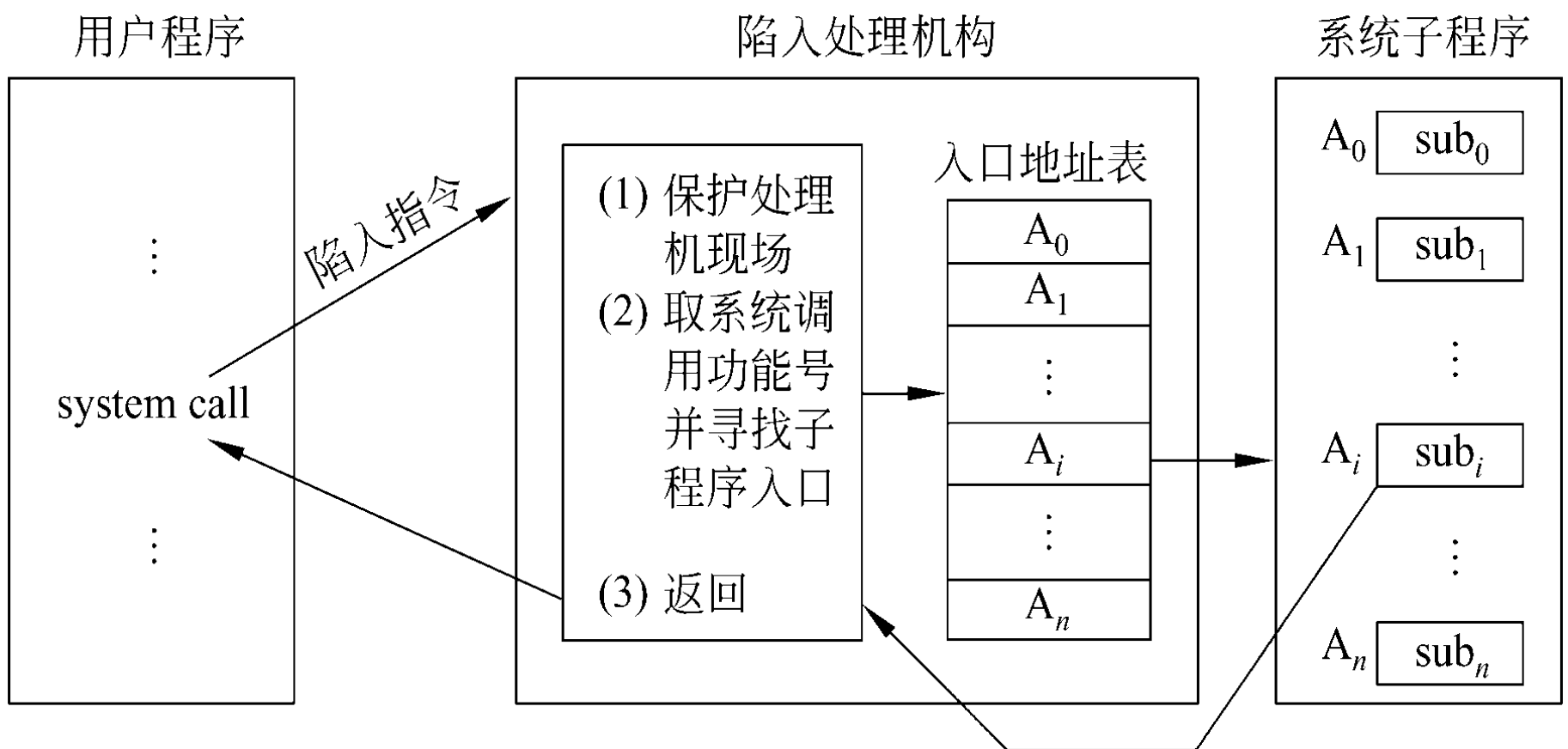
3.1.3.6 自愿性中断的处理

系统调用驱动表：(table driven)

访管号：	服务程序入口
0	addr_0
.....	
m-1	addr_{m-1}

Eg. UNIX

3.1.3.6 自愿性中断的处理



系统调用的处理过程

3.1.3.6 自愿性中断的处理

- 系统调用
 - 与文件有关的系统调用：如创建文件、撤销文件等
 - 与进程有关的系统调用，如创建进程、撤销进程等
 - 与通信有关的系统调用，如发送消息、接收消息
 - 与资源有关的系统调用，如申请资源、释放资源等

- 思考：用户程序只能在目态下运行，并且不能使用特权指令。如果用户程序想要启动外设，或者要完成在目态下无法完成的工作，该怎么办？

提纲

3.1 中断与中断系统

3.2 处理器调度

3.3 调度级别与多级调度

3.4 实时调度

3.5 多处理器调度

3.6 系统举例

3.2 处理器调度

- 3.2.1 处理器调度算法
 - 按什么原则分配
- 3.2.2 处理器调度时机
 - 何时重新分配
- 3.2.3 处理器调度过程
 - 如何完成分配

scheduling

3.2.1 处理器调度算法

- 考虑因素 (scheduling criteria)
 - CPU利用率 ; (max)
 - 吞吐量 ; (max)
 - 周转时间 ; (min)
 - 响应时间 ; (min)
 - 系统开销 ; (min)

调度参数

周转时间：完成时间-进入时间

$$T = t_f - t_s$$

平均周转时间：周转时间的平均值

$$\bar{T} = \frac{1}{n}(\sum_{i=1}^n T_i)$$

带权周转时间：周转时间/运行时间

$$W = \frac{T}{R}$$

平均带权周转时间：带权周转时间的平均值

$$\bar{W} = \frac{1}{n}(\sum_{i=1}^n W_i) = \frac{1}{n}(\sum_{i=1}^n \frac{T_i}{R_i})$$

CPU burst vs. I/O burst

- 阵发期
 - CPU burst cycle: 进程(线程)一次连续使用CPU做计算
 - I/O burst cycle: 进程(线程)一次使用设备做I/O操作
- 进程运行行为
 - CPU burst, I/O burst, CPU burst, I/O burst,
- CPU调度: 考虑处于CPU burst进程集合

CPU burst vs. I/O burst

- 下一个CPU burst的长度估算
 - 令 τ_n 是估计的第 n 个CPU阵发期的长度, t_n 的值是进程最近一次CPU阵发期长度, 则有如下估算公式

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

- 参数 $\alpha(0 \leq \alpha \leq 1)$ 控制 t_n 和 τ_n 在公式中起的作用: 当 $\alpha=0$ 时, $\tau_{n+1}=\tau_n$; 当 $\alpha=1$ 时, $\tau_{n+1}=t_n$ 。通常 α 取0.5

$$\tau_{n+1} = 1/2 t_n + 1/4 t_{n-1} + 1/8 t_{n-2} + 1/16 t_{n-3} + \dots$$

剥夺式调度与非剥夺式调度

- 剥夺式 (preemptive)
 - 就绪进程可以从运行进程手中抢占CPU
 - 获得处理机的进程有3种
 - 进程运行, 直到结束
 - 等待
 - 被抢先
- 非剥夺式 (non-preemptive)
 - 就绪进程不可从运行进程手中抢占CPU
 - 获得处理机的进程有2种
 - 进程运行, 直到结束
 - 等待

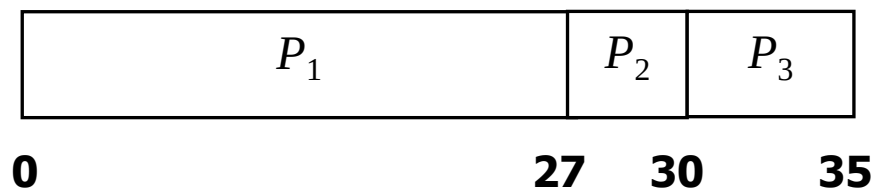
3.2.1.1 先到先服务算法

- FCFS (First Come First Serve)

- 按进程申请CPU（就绪）的次序

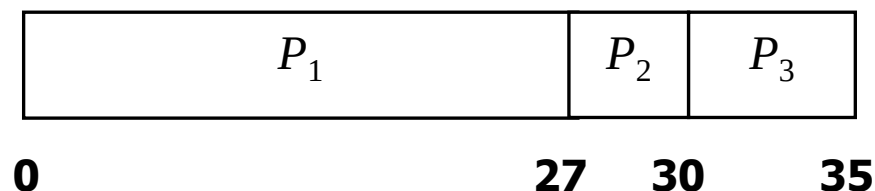
- | <u>Process</u> | <u>Arrival time</u> | <u>Burst time</u> |
|----------------|---------------------|-------------------|
| ■ P_1 | 0 | 27 |
| ■ P_2 | 1 | 3 |
| ■ P_3 | 2 | 5 |

- CPU调度状况可用Gantt图表示



以图示的方式形象地表示出进程的活动顺序与持续时间

3.2.1.1 先到先服务算法(Cont.)



进程	到达时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
P_1	0	27	0	27	27	1
P_2	1	3	27	30	29	9.67
P_3	2	5	30	35	33	6.6

平均周转时间 $\bar{T} = (27 + 29 + 33) / 3 = 29.67$

平均带权周转时间 $\bar{W} = (1 + 9.67 + 6.6) / 3 = 5.76$

3.2.1.1 先到先服务算法(Cont.)

- 优点：
 - “公平”
- 缺点：
 - 短作业等待时间长

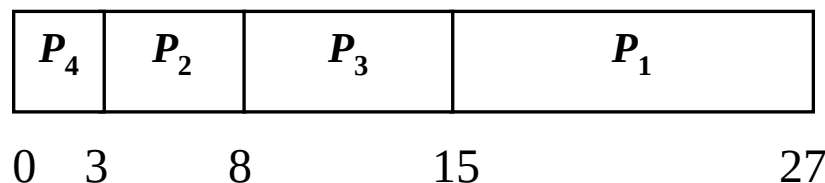
3.2.1.2 短作业优先

- SJF(Shortest Job First)

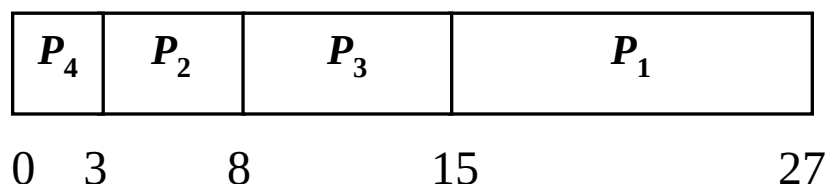
- 按CPU burst长度

Process	Arrival time	Burst time
P_1	0	12
P_2	0	5
P_3	0	7
P_4	0	3

- Gantt Chart



3.2.1.2 短作业优先(Cont.)



进程	到达时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
P_1	0	12	15	27	27	2.25
P_2	0	5	3	8	8	1.6
P_3	0	7	8	15	15	2.14
P_4	0	3	0	3	3	1

平均周转时间 $\bar{T} = (27 + 8 + 15 + 3) / 4 = 13.25$

平均带权周转时间 $\bar{W} = (2.25 + 1.6 + 2.14 + 1) / 4 = 1.75$

3.2.1.2 短作业优先(Cont.)

- 特点：
 - 假定所有任务同时到达，平均等待时间最短
 - 忽略了作业等待时间，长作业可能被饿死

3.2.1.2 短作业优先(Cont.)

■ 例：

- 现有三个同时到达的作业J1，J2和J3，它们的执行时间分别为T1、T2和T3，且 $T1 < T2 < T3$ ，系统按照单道方式运行且采用短作业优先算法，则平均周转时间为_____.

$$(3T1+2T2+T3) / 3$$

3.2.1.3 最短剩余时间优先算法(SRTN)

- Shortest Remaining Time Next
- 可剥夺SJF，选择剩余时间最短的进程或线程

■ Process Arrival time Burst time

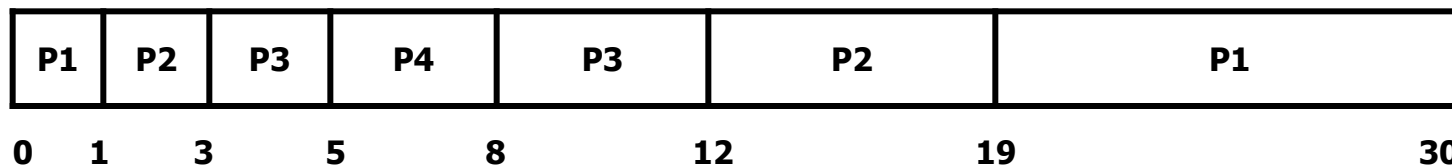
■ P1 0 12

■ P2 1 9

■ P3 3 6

■ P4 5 3

■ Gantt图



3.2.1.3 最短剩余时间优先算法(Cont.)

P1	P2	P3	P4	P3	P2	P1	
0	1	3	5	8	12	19	30

进程	到达时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
P ₁	0	12	0	30	30	2.5
P ₂	1	9	1	19	18	2
P ₃	3	6	3	12	9	1.5
P ₄	5	3	5	8	3	1

平均周转时间=(30+18+9+3)/4=15

平均带权周转时间=(2.5+2+1.5+1)/4=1.75

平均等待时间=(18+9+3+0)/4=7.5(ms)

3.2.1.4最高响应比优先(HRN)

- Highest Response Ratio Next
 - $RR = (BT + WT) / BT = 1 + WT / BT$
 - BT=burst time
 - WT=wait time
 - 计算每个作业的响应比，选择响应比最高的作业优先投入运行
 - 优点
 - 同时到达任务，短者优先
 - 长作业随等待时间增加响应比增加

3.2.1.4最高响应比优先(Cont.)

- 例

- 一个作业8:00到达系统, 估计运行时间为1小时。若从10:00开始执行, 则其响应比为_____。

$$RR = (BT + WT) / BT = 1 + WT / BT = 1 + 2 / 1 = 3$$

3.2.1.5 最高优先数算法(HPF)

- 静态优先数 (static)
 - 在进程创建时分配，生存期内不变
 - 简单，开销小
 - 公平性差，可能会造成低优先数进程长期等待
 - 适合批处理进程
- 动态优先数 (dynamic)
 - 进程创建时继承优先数，生存期内可以修改
 - 获得资源优先数提高
 - 就绪时随等待处理机时间增长而提高
 - 资源利用率高，公平性好
 - 开销大，实现复杂

适用于实时系统

3.2.1.5 最高优先数算法(Cont.)

- 非剥夺式

- 获得处理机的进程运行，直至
 - 终止
 - 等待

调度时机

- 剥夺式数

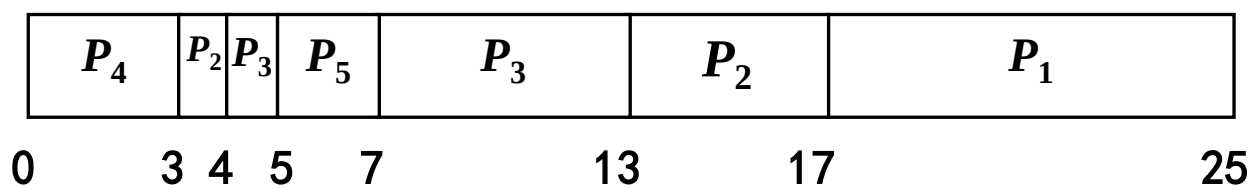
- 获得处理机的进程运行，直至
 - 终止
 - 等待
 - 出现高优先级的进程（新创建或被唤醒的进程）

3.2.1.5 最高优先数算法(Cont.)

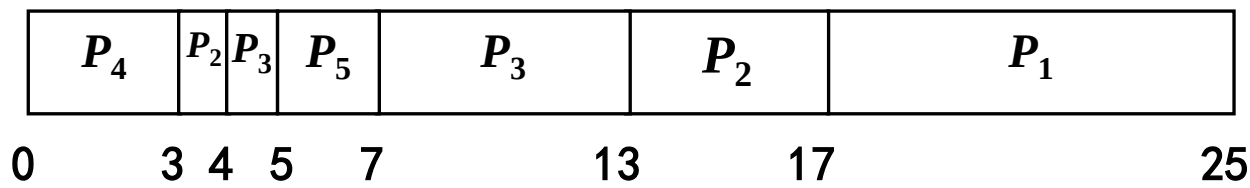
- 可抢占CPU

Process	Arrival time	Priority	Burst time
■ P1	0	0	8
■ P2	2	1	5
■ P3	4	3	7
■ P4	0	2	3
■ P5	5	7	2

- Gantt Chart



3.2.1.5 最高优先数算法(Cont.)



进程	到达时间	运行时间	优先级	开始时间	完成时间	周转时间	带权周转时间
P_1	0	8	0	17	25	25	3.13
P_2	2	5	1	3	17	15	3
P_3	4	7	3	4	13	9	1.29
P_4	0	3	2	0	3	3	1
P_5	5	2	7	5	7	2	1

平均周转时间 $\bar{T} = (25 + 15 + 9 + 3 + 2) / 5 = 38.8$

平均带权周转时间 $\bar{W} = (3.13 + 3 + 1.29 + 1 + 1) / 5 = 1.88$

3.2.1.5 最高优先数算法(Cont.)

- 例子UNIX: preemptive+dynamic priority（可抢占CPU动态优先数）。
 - 计算公式: $p_pri = \min\{127, \text{USER} + p_cpu/16 + p_nice\}$
 - 定义USER=100;
 - p_cpu: 运行进程每20ms加1（优先级降低），其它进程每1200ms减10（优先级提高）；
 - p_nice: 可以通过系统调用nice(...)修改的量：规定用户进程0~20之间（低），系统进程-20~+20之间（高）。
 - 调度时取p_pri最小的。

3.2.1.6 循环轮转算法(RR)

- Round Robin (RR)

- 系统为每个进程规定一个时间片，所有进程按照其时间片长短轮流的运行

- 基本轮转

- 时间片 (quantum, time slice) 长度固定，不变
- 所有进程等速向前推进

- 改进轮转

- 时间片长度不定，可变

适用于分时系统

3.2.1.6 循环轮转算法 (Cont.)

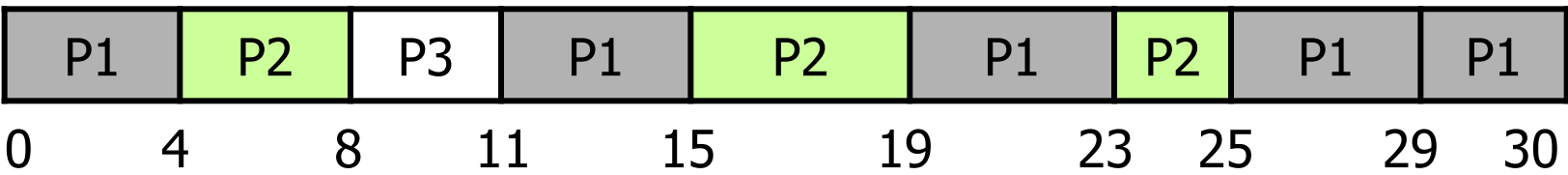
- 时间片长度
 - 几十毫秒~几百毫秒 (eg. 50ms)
 - 过长：响应速度慢
 - 过短：系统开销 (overhead) 大

3.2.1.6 循环轮转算法 (Cont.)

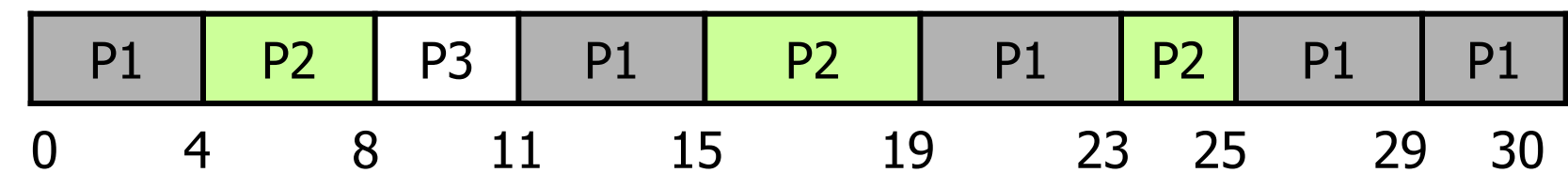
- RR可抢占CPU调度 : time slice=4ms

<u>Process</u>	<u>Arrival time</u>	<u>Burst time</u>
■ P1	0	17
■ P2	0	10
■ P3	0	3

- Gantt Chart



3.2.1.6 循环轮转算法 (Cont.)



进程	到达时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
P ₁	0	17	0	30	30	1.76
P ₂	0	10	4	25	25	2.5
P ₃	0	3	8	11	11	3.67

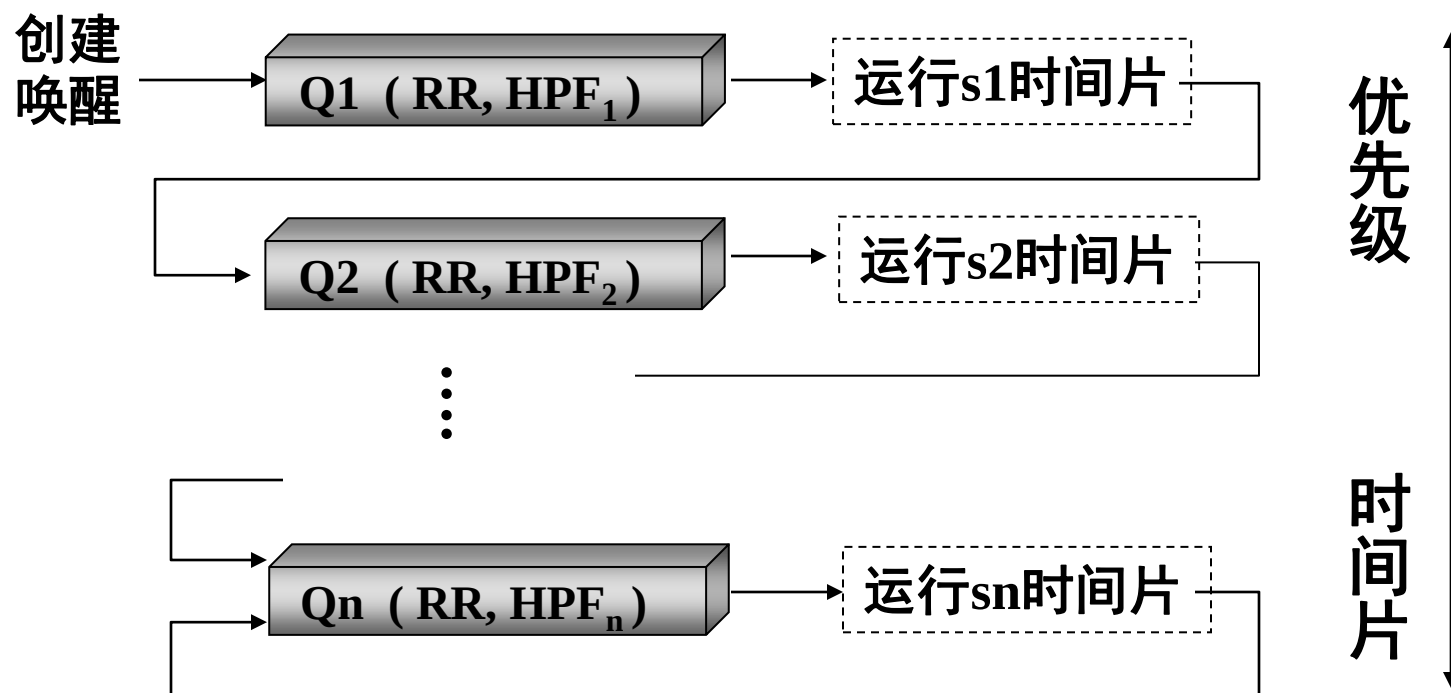
平均周转时间 $= (30 + 25 + 11) / 3 = 22$
 平均带权周转时间 $= (1.76 + 2.5 + 3.67) / 3 = 2.64$
 平均等待时间 $= (13 + 15 + 8) / 3 = 12(\text{ms})$

3.2.1.7 多级队列算法(MLQ)

- 多级队列
 - 多个就绪队列，进程所属的队列固定
 - 每个队列可以采用不同的调度算法
- 例如：通用系统中
 - 队列1：实时进程就绪队列 (HPF)
 - 队列2：分时进程就绪队列 (RR)
 - 队列3：批处理进程就绪队列 (HPF)
- 当CPU空闲时
 - 优先选择队列1中进程。如果队列1为空，则选择队列2中进程，否则选择队列3中进程

3.2.1.8 反馈排队算法 (FB)

- Feed-Back:
 - 多个就绪队列，进程所属队列可变



3.2.1.8 反馈排队算法 (Cont.)

- 调度效果
 - 资源利用率高
 - P1等待P2占有的资源R, P2释放R, 分给P1, P1被唤醒, 进入高优先级队列, 可尽早投入运行, 使用资源R
 - 响应速度快
 - 交互式进程经常进入等待状态(等待用户输入), 一旦被唤醒(输入完成), 进入高优先级队列, 可尽快被调度选中, 投入运行, 反应及时
 - 系统开销小
 - 计算量大的进程用完前面 $n-1$ 级时间片, 没有处理完, 落入底层队列, 调度频率下降, 但每次获得较长的时间片

Feed Back

3.2.1.8 反馈排队算法 (Cont.)

- 缺点：处罚长进程，长进程可能被饿死
- 优点：不需要计算进程执行所需时间

3.2.2 处理器调度时机

- 运行进程结束
- 运行进程等待
- 处理器被剥夺

中断与处理器切换的关系

- 中断是处理器切换的必要条件，但不是充分条件
- 必然引起进程切换的中断
 - 进程自愿结束，`exit()`
 - 进程等待
 - 进程被强行终止；
 - 非法指令，越界
- 可能引起进程切换的中断
 - 时钟
 - 系统调用，`read(fd, buf, 1024)`

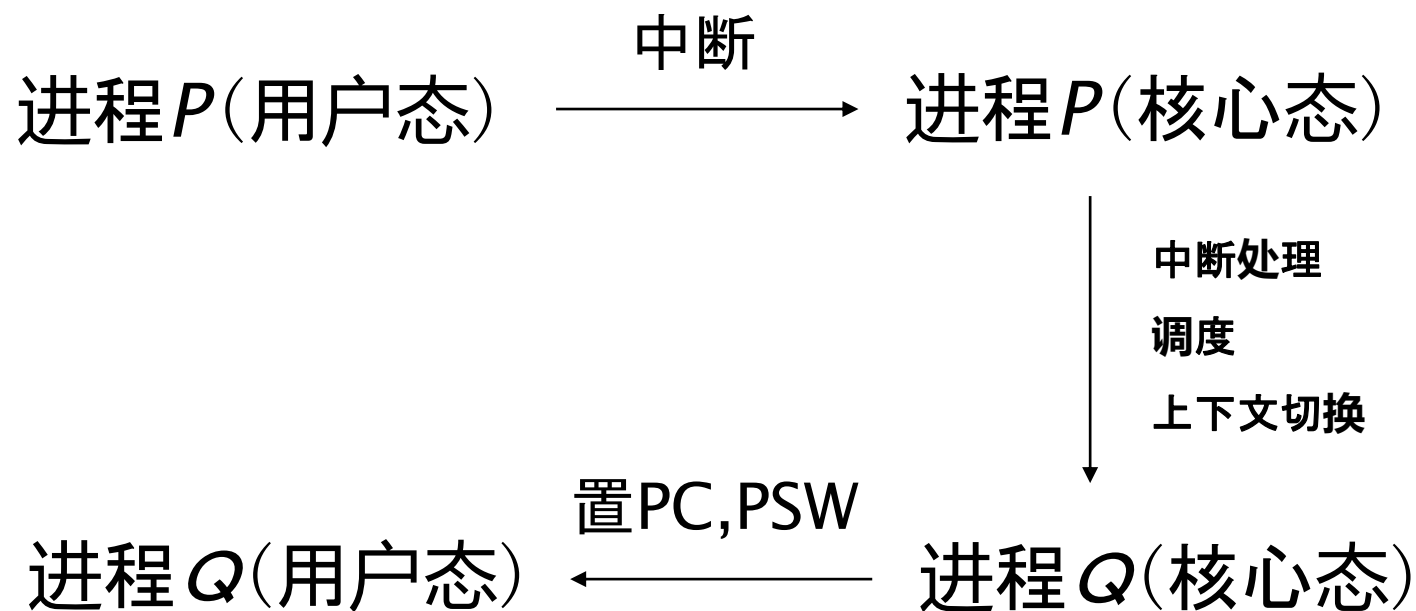
3.2.3 处理机调度过程

- 进程切换
 - 中断事件处理过程完后，如果需要进行进程切换，则转到处理器分派程序dispatcher，选择一个上升进程运行

3.2.3 处理机调度过程

- dispatcher
 - 保存下降进程的现场
 - 寄存器 (PSW, PC, SP, 通用寄存器, 地址寄存器) $\Rightarrow$ PCB
 - 选择上升进程
 - 按处理机调度算法在就绪队列中选择一个进程
 - 为防止就绪队列为空, 系统通常安排一个“闲逛”进程, 该进程永远也运行不完, 且调度级别最低, 当系统中无其它进程时, 运行“闲逛”进程
 - 恢复上升进程的现场
 - PCB $\Rightarrow$ 寄存器
 - 先恢复通用寄存器和地址寄存器, 最后恢复PSW, PC
 - PSW和PC必须用一条指令恢复

UNIX进程调度实现



提纲

3.1 中断与中断系统

3.2 处理器调度

3.3 调度级别与多级调度

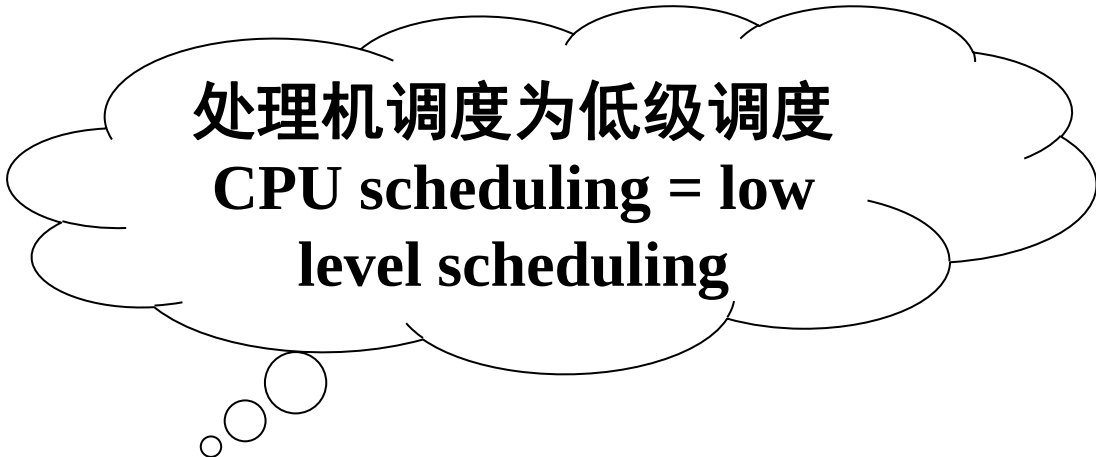
3.4 实时调度

3.5 多处理器调度

3.6 系统举例

3.3 调度级别与多级调度

- 3.3.1 交换与中级调度
 - Swapping and mid-level scheduling
- 3.3.2 作业与高级调度
 - Job and high-level scheduling



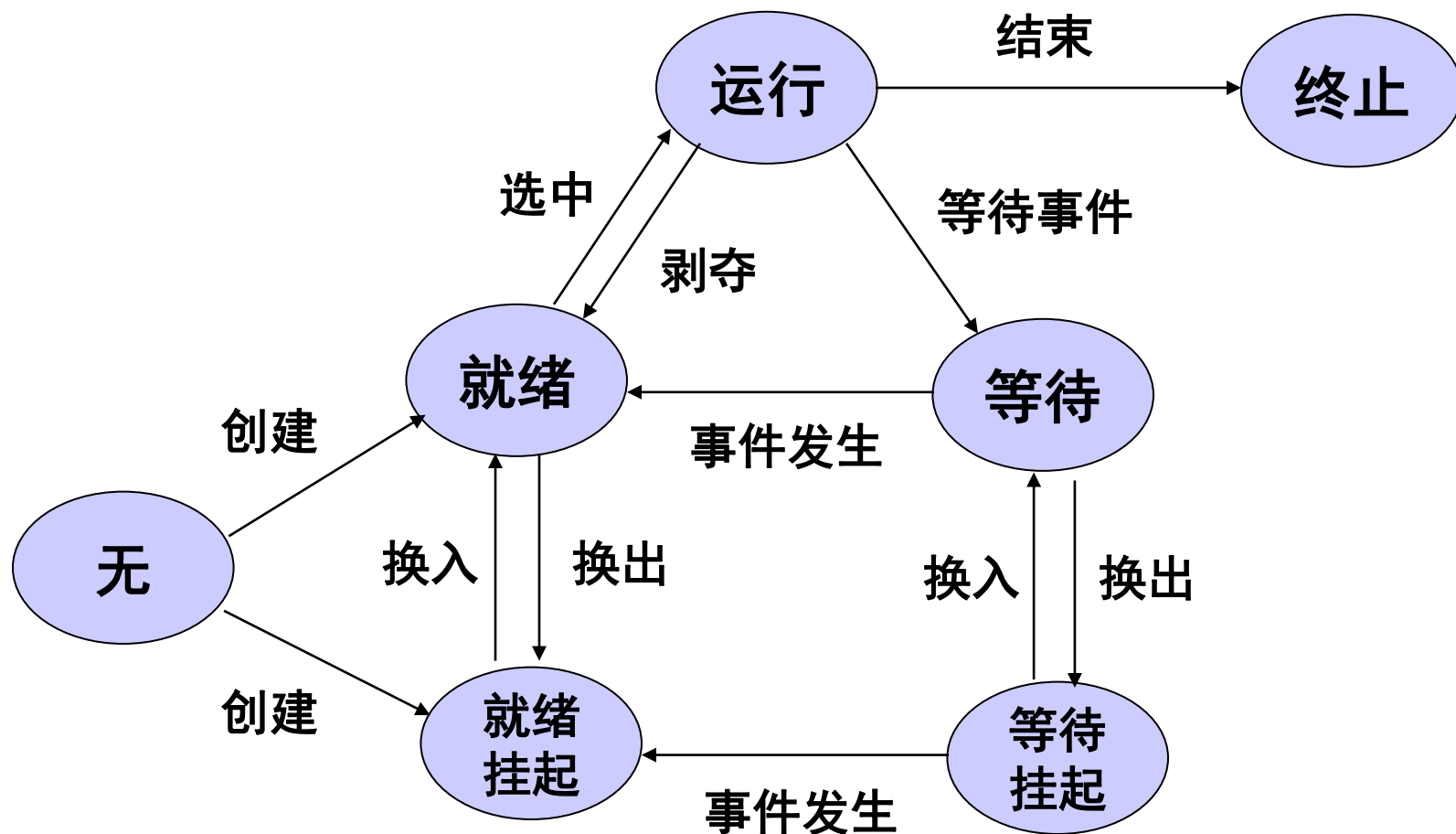
处理机调度为低级调度
CPU scheduling = low level scheduling

-
- 调度的层次
 - 高级调度
 - 中级调度
 - 低级调度

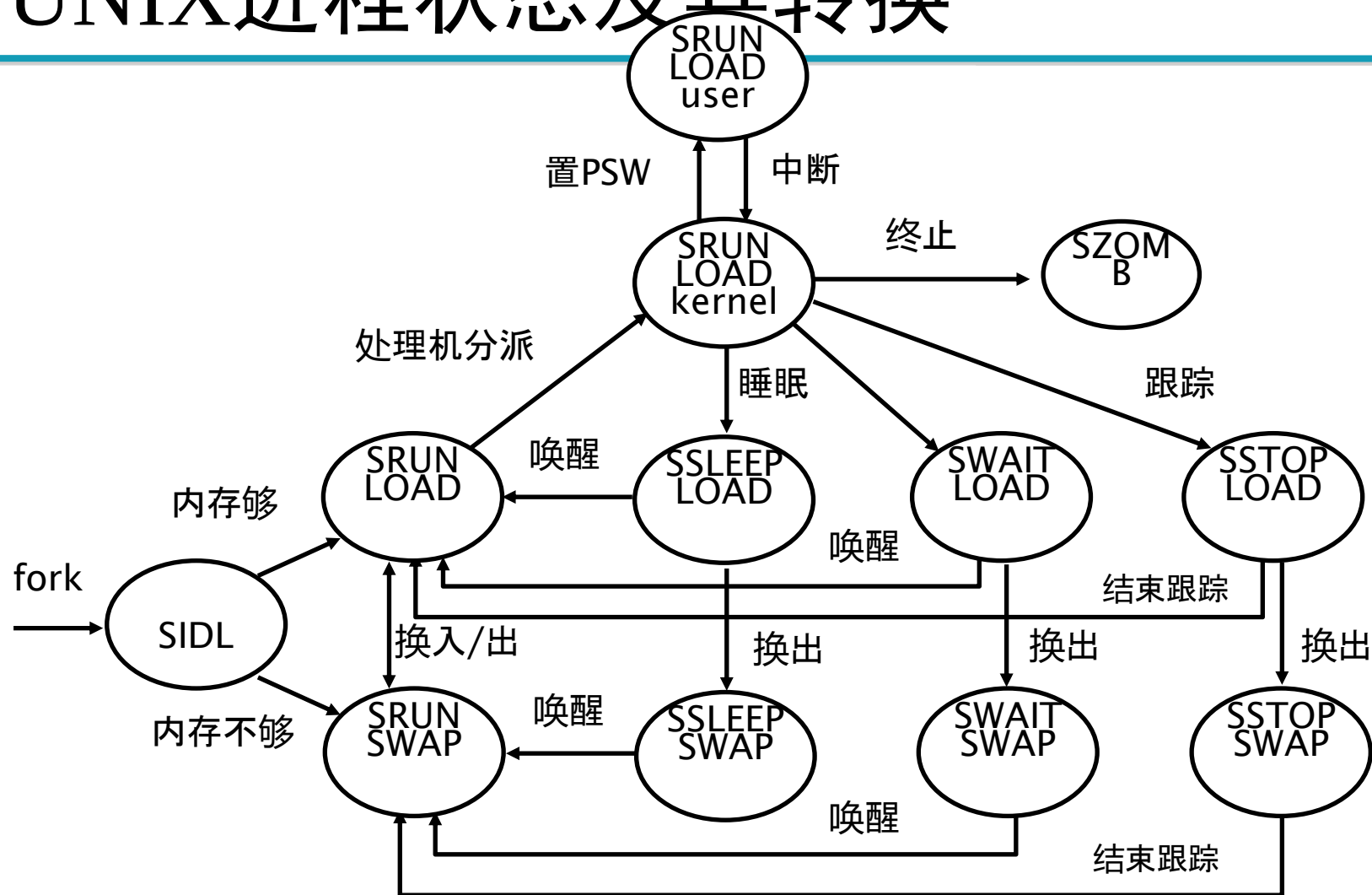
3.3.1 交换与中级调度

- 术语
 - 交换(swapping)：进程在内存和外存储器之间的调度
 - 中级调度(mid-level scheduling)：是系统控制并发程度的一个级别
 - 并发度(degree of multi-programming)：同时向前推进的进程个数
- 交换目标
 - 减少并发度以降低系统开销
 - 缓解内存空间资源紧张的矛盾
- 并发度过高
 - 系统开销大
 - 响应速度慢
 - 内存等资源紧张
 - 进程(线程)频繁进入等待状态
 - More deadlocks

3.3.1 交换与中级调度



UNIX进程状态及其转换



UNIX的中级调度 (sched #0)

- 移入SRUN状态进程
- 如内存不够
 - 移出SWAIT和SSTOP状态进程；
 - 如还不够，移出SSLEEP和SRUN状态进程
 - 条件
 - 待移入进程在外存时间 ≥ 3 秒
 - 待移出进程在内存时间 ≥ 2 秒

3.3.2 作业与高级调度

- 作业调度
 - 将一个作业由（磁盘的）输入并调入内存，并为其建立相应的进程，使其具有运行的资格
- 批处理作业的处理结果
 - 程序的执行结果
 - 计帐信息

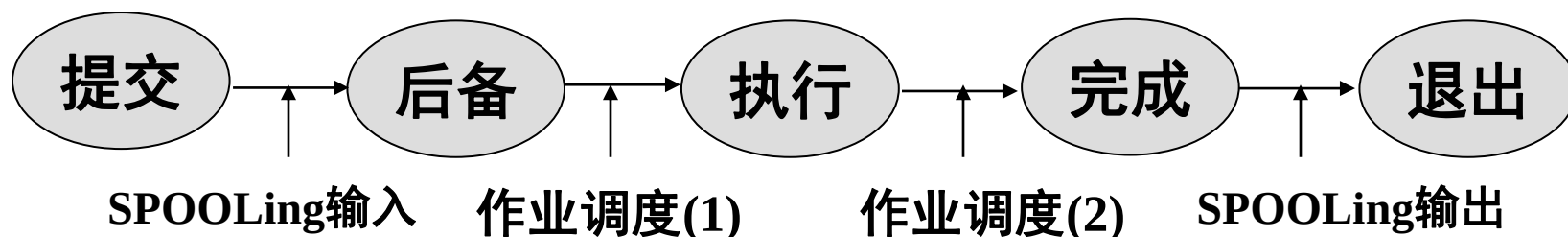
3.3.2 作业与高级调度

■ 作业状态

- 提交：输入机向输入井传送
- 后备：在输入井, 尚未进入内存
- 执行：分解为进程, 在内存处理
- 完成：处理完毕, 结果在输出井
- 退出：由输出井向打印机传送

3.3.2 作业与高级调度

- 状态转换
 - 提交→后备：由SP00Ling输入进程完成
 - Simultaneous Peripheral Operation On-Line
 - 后备→执行：由作业调度(1)进程完成
 - 高级调度：系统进程
 - 执行→完成：由作业调度(2)完成
 - 完成→退出：由SP00Ling输出进程完成



作业控制块与作业表

- JCB (Job Control Block)

- 作业标识
- 所属用户
- 作业状态
- 调度参数
- 输入井地址
- 输出井地址
- 资源需求
- 进入时间
- 处理时间
- 完成时间

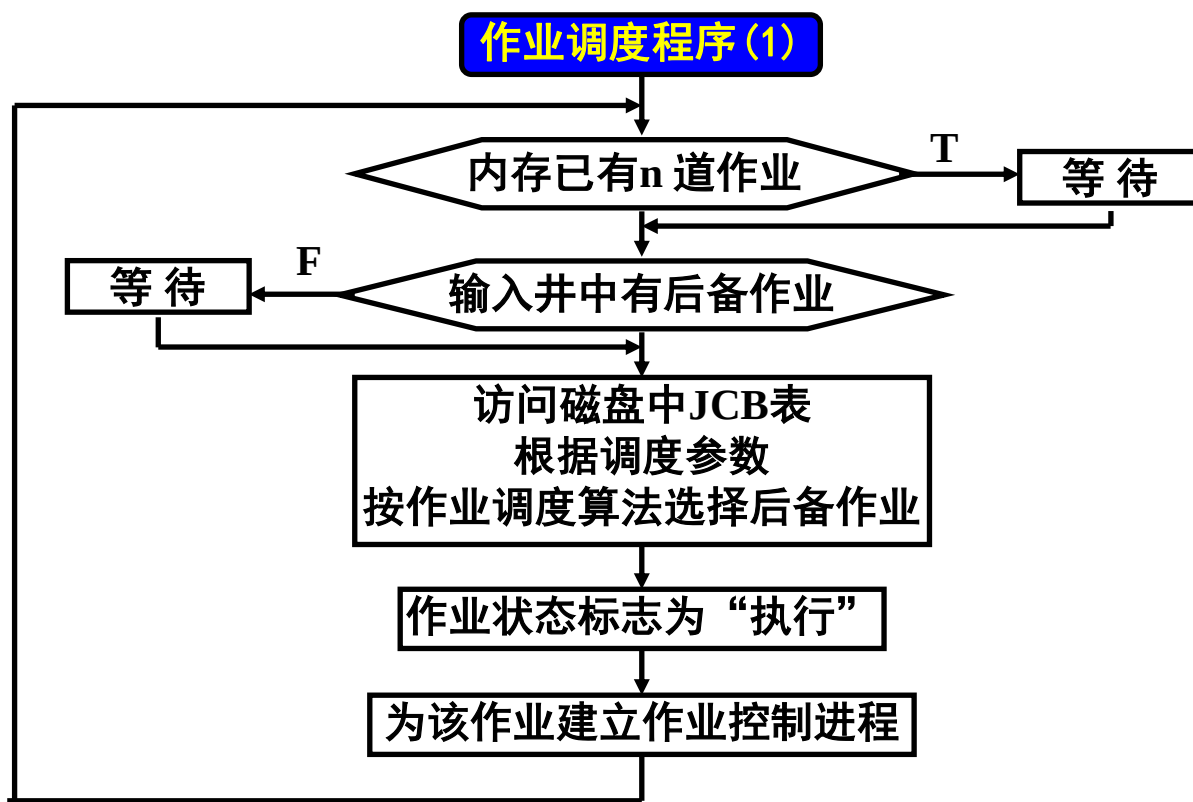
作业表

JCB1
JCB2
JCB3
...
JCBk

- SP00ling输入建立，作业调度使用，SP00ling输出撤销

3.3.2 作业与高级调度(Cont.)

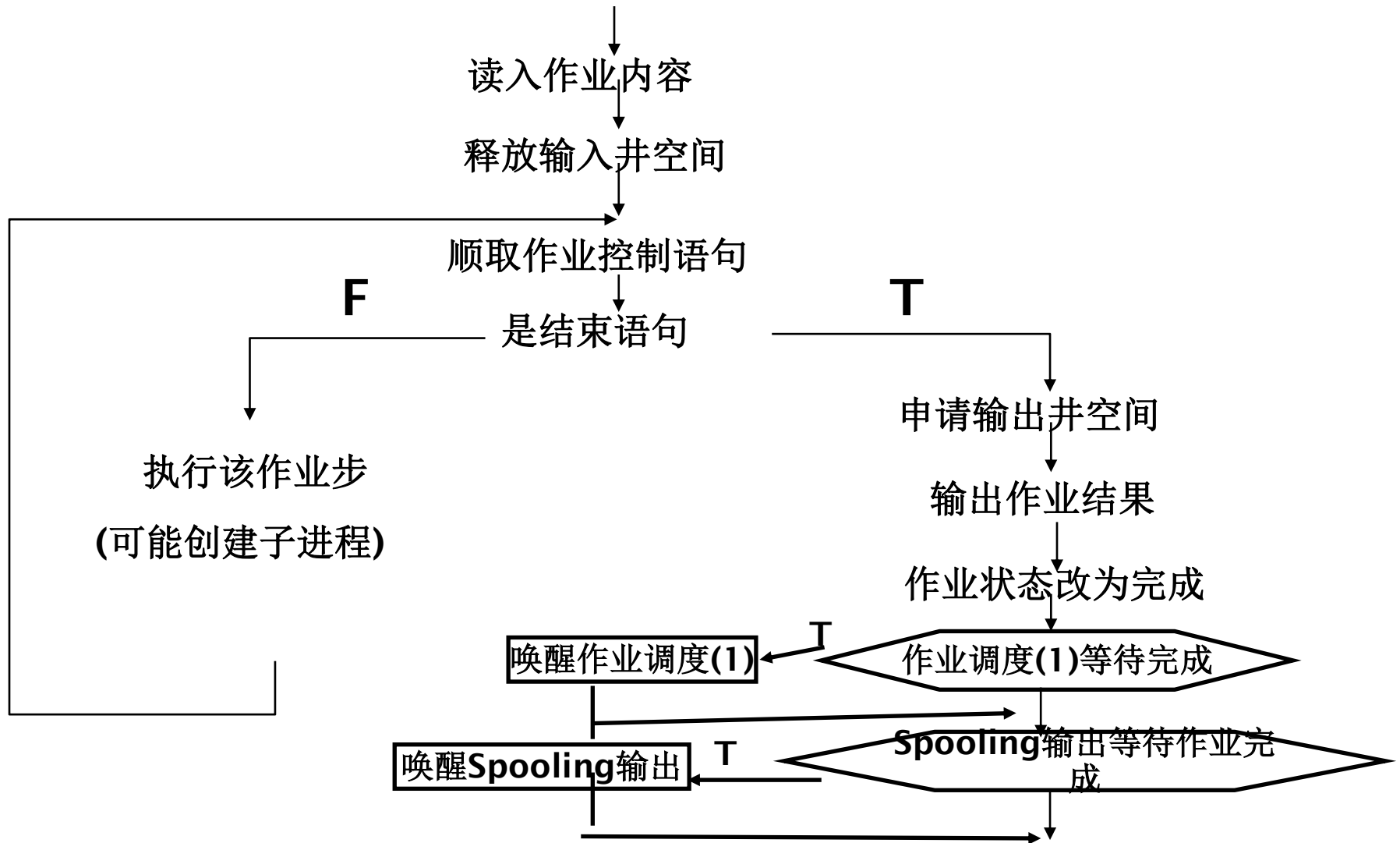
1. 批处理作业调度程序(1)



在后备作业集合中选择作业，并为其建立作业控制进程来处理该作业。

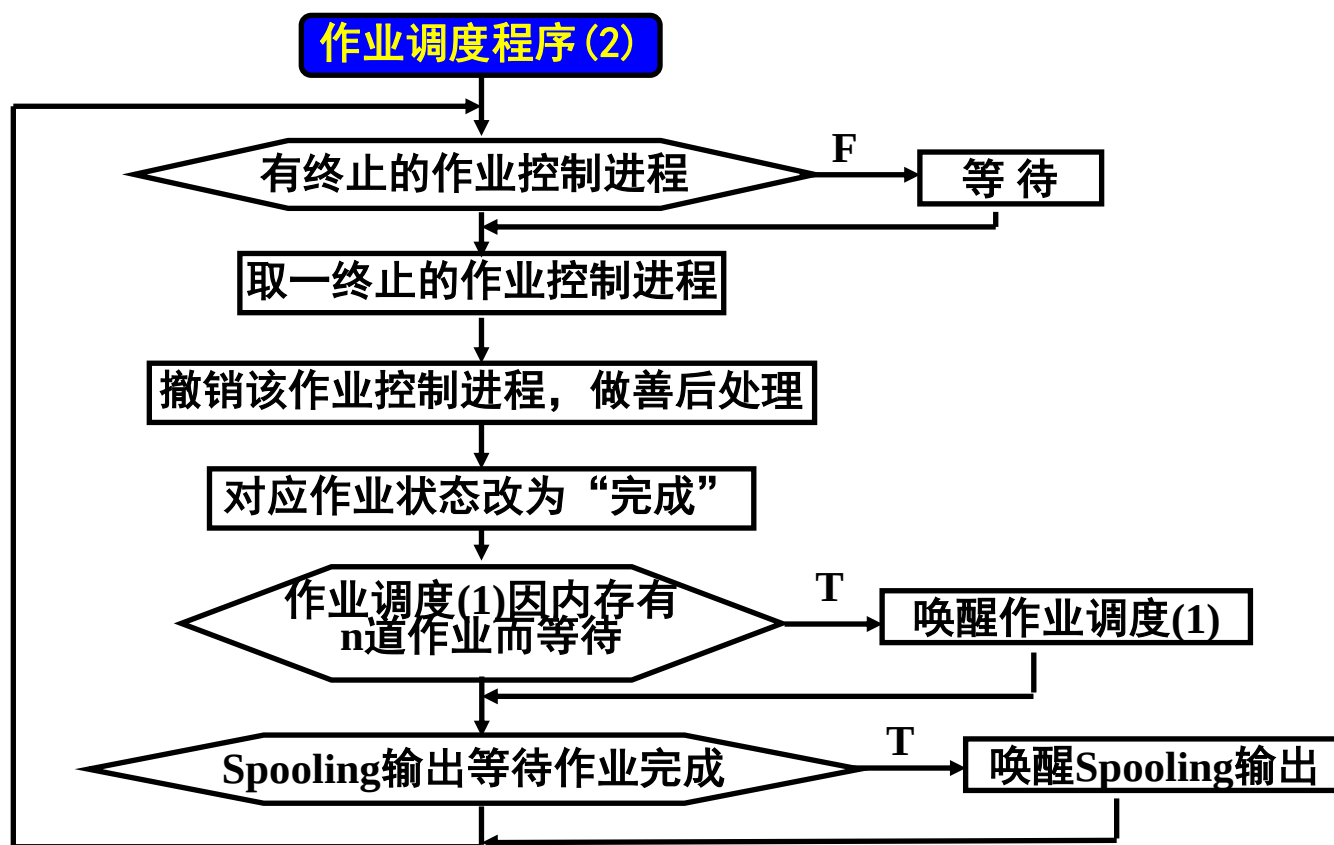
3.3.2 作业与高级调度(Cont.)

作业控制进程



3.3.2 作业与高级调度(Cont.)

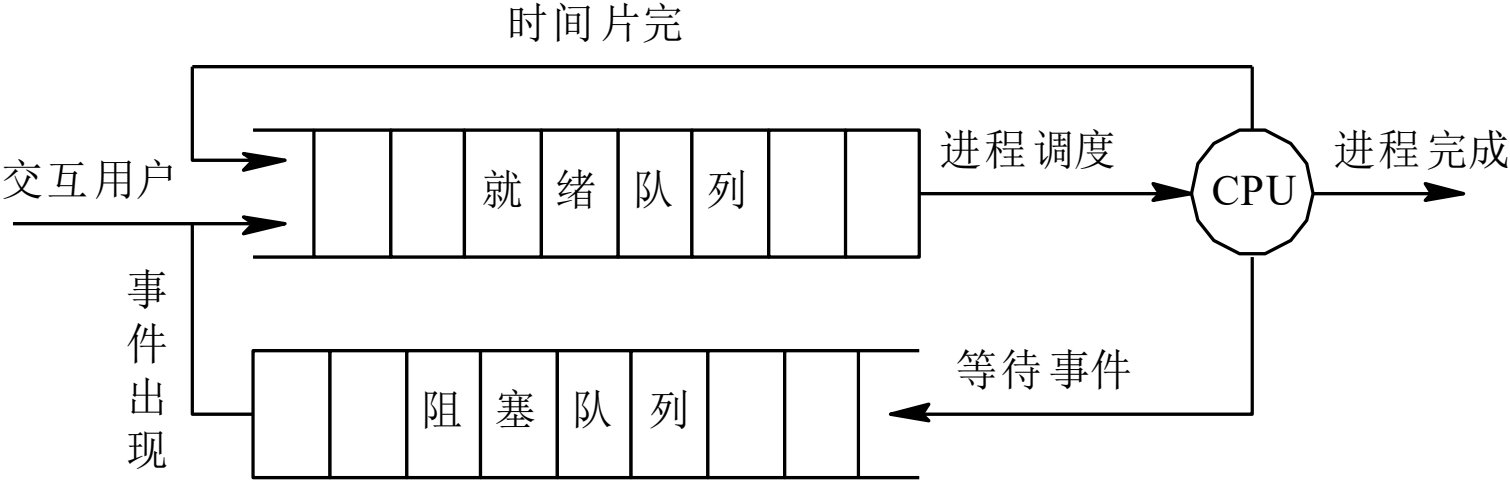
2. 批处理作业调度程序(2)



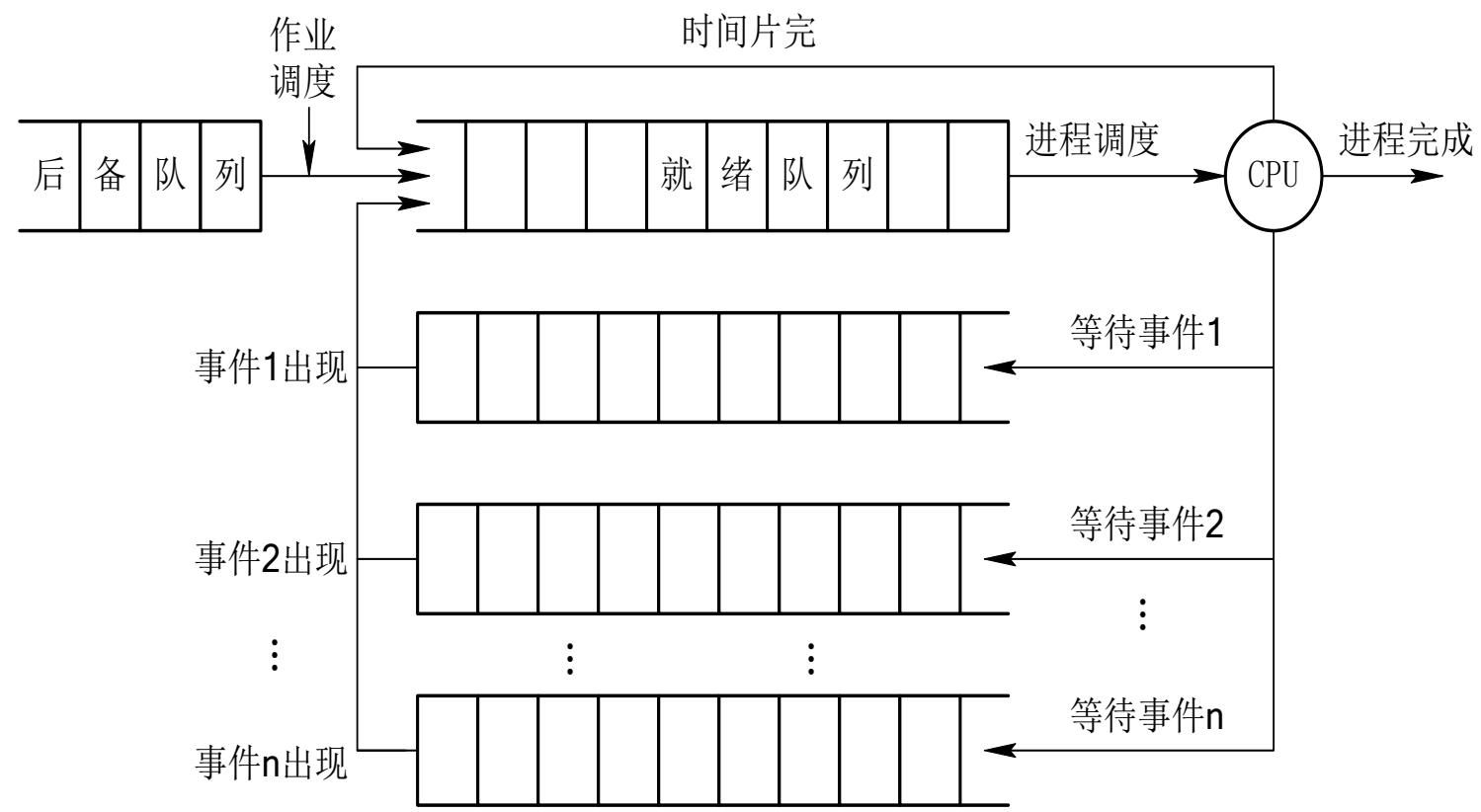
作业调度算法

- 适合批作业调度的算法
 - 先到先服务算法 (FCFS)
 - 优先数调度算法 (HPF)
 - 短作业优先调度算法 (SJF)
 - 最高响应比优先调度算法 (HRN)
- 不适合批作业调度的算法
 - 时间片轮转算法 (RR)
 - 最短剩余时间优先 (SRTN)
 - 反馈排队算法 (FB)

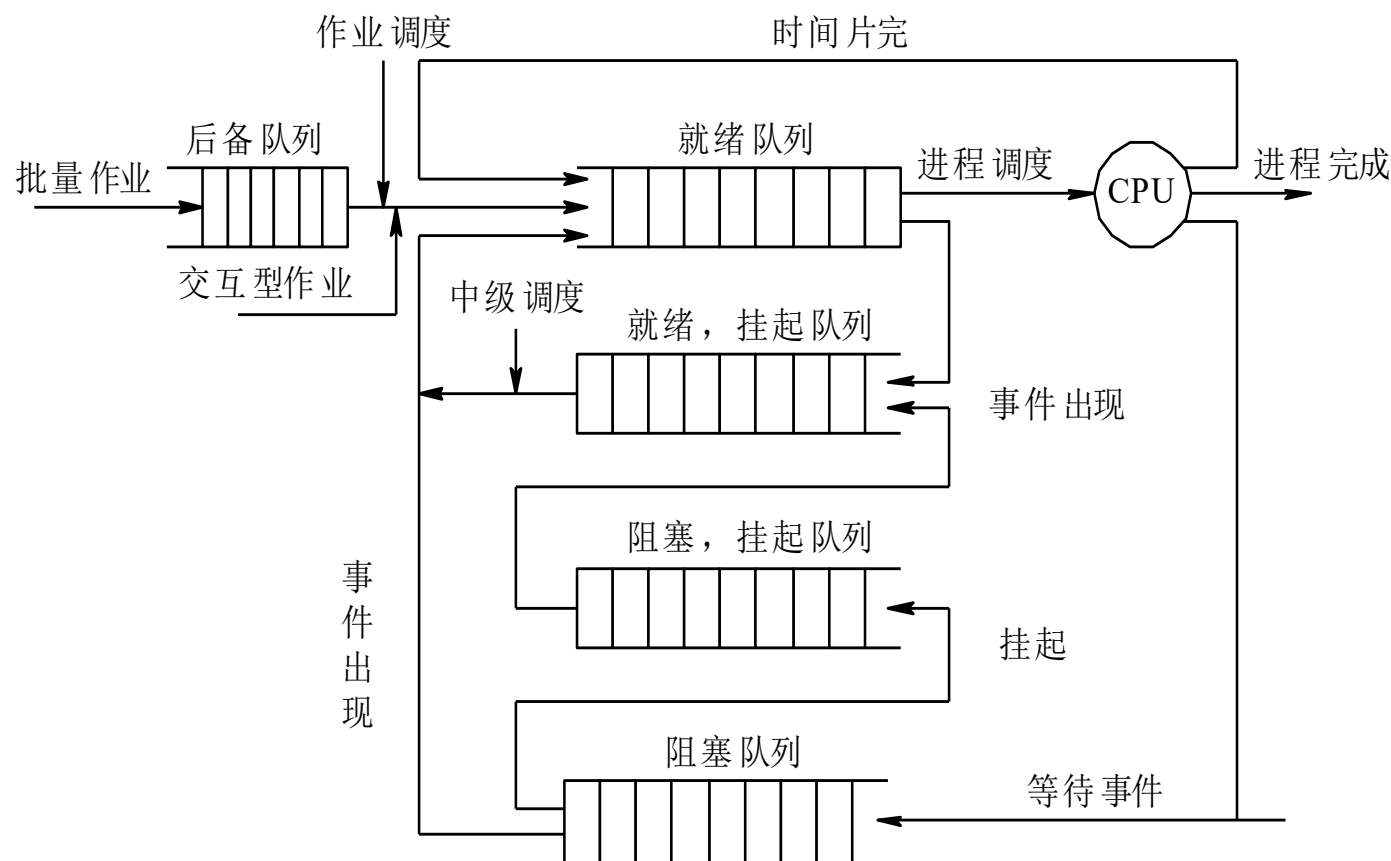
仅有进程调度的调度队列模型



具有高级调度和低级调度的调度队列模型



同时具有三级调度的调度队列模型



提纲

3.1 中断与中断系统

3.2 处理器调度

3.3 调度级别与多级调度

3.4 实时调度

3.5 多处理器调度

3.6 系统举例

3.4 实时调度(real-time scheduling)

- 实时任务：
 - 具有明确时间约束的计算任务
 - Eg.
 - 某时刻前必须开始处理
 - 某时刻前必须处理完毕
- 实时调度：
 - 合理安排就绪实时任务的执行次序，满足每个实时任务时间约束条件的调度

实时任务分类

- 硬实时 vs. 软实时
 - 硬实时(hard real-time): 必须满足任务截止期要求
 - 软实时(soft real-time): 期望满足截止期要求
- 周期性 vs. 随机性
 - 周期性: 每隔固定时间发生一次
 - 随机性: 由随机事件触发, 其发生时刻不确定

术语解释

- 就绪时间 (Ready time) : 实时任务产生并可以开始处理的时间
- 开始截止期 (Starting deadline) : 实时任务最迟开始处理的时间
- 处理时间 (Processing time) : 实时任务处理所需要的处理器时间
- 完成截止期 (Completion deadline) : 实时任务最迟完成时间
- 发生周期 (Occurring period) : 发生周期性实时任务的间隔时间

周期性实时事务

- 周期性实时事务:
 - 令 C_i 为任务 P_i 处理时间, T_i 为任务 P_i 的发生周期, 则任务 $P_1, \dots, P_m$ 可调度的必要条件为:

$$\sum_{i=1}^m \frac{C_i}{T_i} \leq 1$$

周期性实时事务

- 例：

- $T1=100, T2=200, T3=500$ (ms)
- $C1=50, C2=30, C3=100$ (ms)
- $C1/T1+C2/T2+C3/T3=0.5+0.15+0.2=0.85<1$
- 满足可调度的必要条件

周期性实时事务

例：

进程	就绪时间	处理时间	完成截止期	发生周期
A	0	10	20	20
B	0	25	50	50

$10/20 + 25/50 = 1$, 可调度(不考虑开销)

例子

Process	Arrival time	Execution time	Completion deadline
A(1)	0	10	20
A(2)	20	10	40
A(3)	40	10	60
A(4)	60	10	80
A(5)	80	10	100
.....			
B(1)	0	25	50
B(2)	50	25	100

非周期性实时任务

- 非周期性实时任务：
 - 若实时任务到达服从参数为 λ 的泊松分布时，当 $1/\lambda$ 值很小，小于等于一个阈值，则可以将其视为周期性实时任务调度，任务的平均到达时间间隔为 $1/\lambda$ ——平均周期
- 实时调度有剥夺式和非剥夺式之分
 - 剥夺式：调度灵活，在满足时间约束条件下可以充分利用处理器，实现复杂
 - 非剥夺式：简单，为满足约束条件，处理器往往不能满负载运行。

3.4.1 最早截止期调度

- EDF(Earliest Deadline First)
 - 优先选择完成截止期最早的实时任务
 - 对于新到达的任务, 如果其完成截止期先于正在运行任务的完成截止期, 则重新分派处理器, 即剥夺
- 对EDF来说, 可调度条件是:

$$\sum_{i=1}^m \frac{C_i}{T_i} \leq 1$$

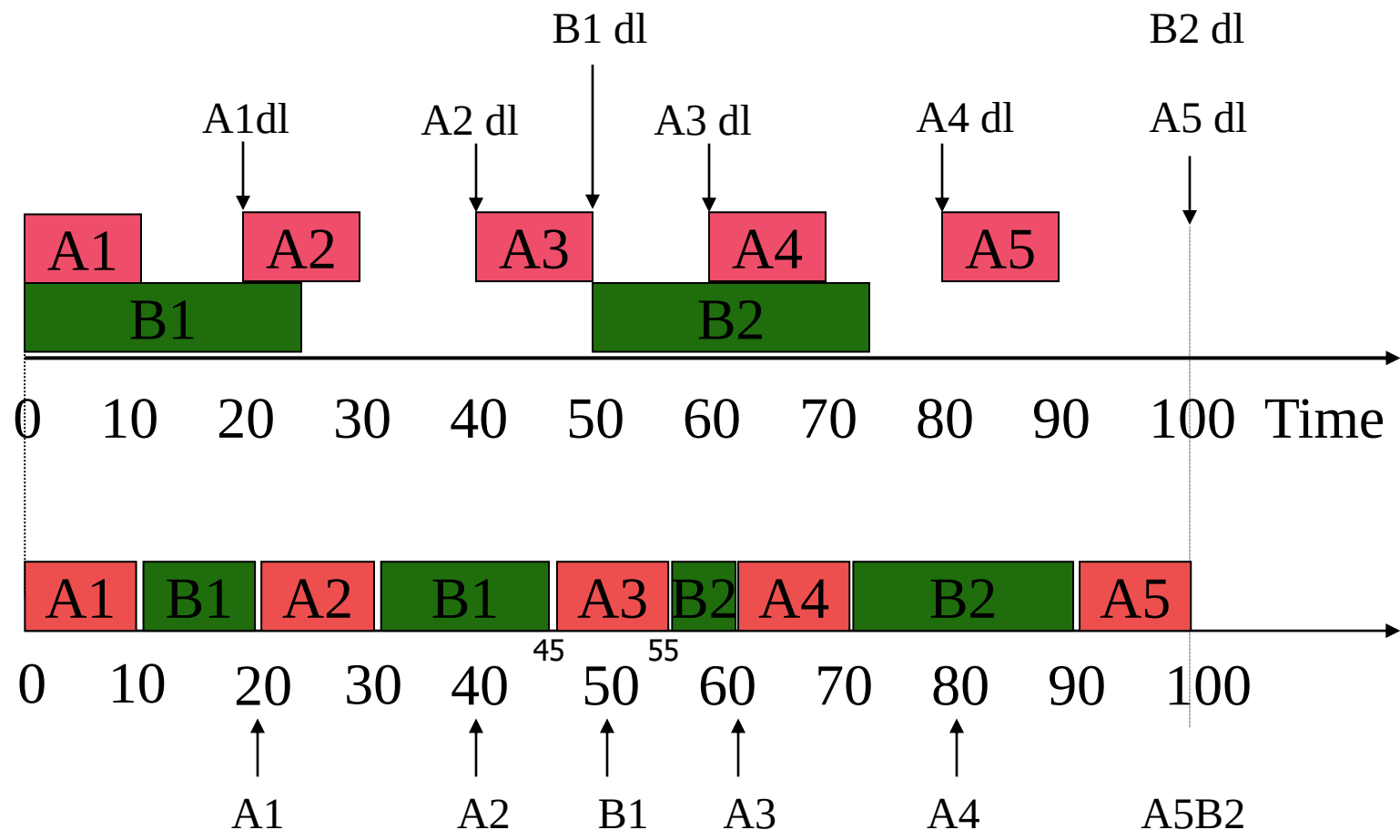
- C_i 表示任务 P_i 的处理时间, T_i 表示任务 P_i 的发生周期

例：

进程	就绪时间	处理时间	完成截止期	发生周期
A	0	10	20	20
B	0	25	50	50

$10/20 + 25/50 = 1$, 可调度(不考虑开销)

例子: Earliest Deadline First



3.4.2 速率单调调度

- RMS (Rate Monotonic Scheduling)
 - 提出于1973年
 - 面向周期性实时事务，非剥夺式
 - 优先调度发生周期最短（频度最高）的实时任务
 - 可调度条件：

$$\sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

RMS的上限值

n	$n(2^{1/n} - 1)$
1	1.0
2	0.828
3	0.779
4	0.756
5	0.743
6	0.734
⋮	⋮
∞	$\ln 2 \approx 0.693$

RMS vs. EDF

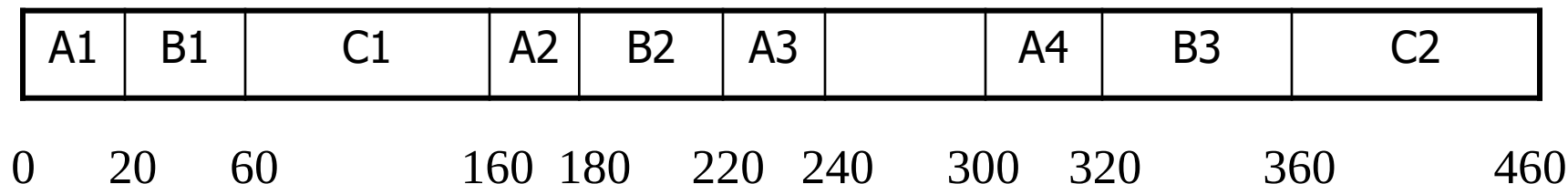
- 1) RMS可调度条件强于EDF
- 2) RMS调度较EDF实现简单

RMS例子：

进程	T_i	C_i
A	100	20
B	150	40
C	350	100

$$\frac{C_1}{T_1} + \frac{C_2}{T_2} + \frac{C_3}{T_3} = 0.2 + 0.267 + 0.286 = 0.753 \leq 3(2^{\frac{1}{3}} - 1) = 0.779$$

可调度，具体调度结果：



3.4.3 最小裕度优先调度

- Minimum Laxity First

- 类似最早截止期优先，优先级基于一个进程的裕度

$$L = D - (T + C)$$

L为裕度，T为当前时间，D为任务的截止期，C为剩余处理时间

- 如果一个任务的裕度为0，则它必须立即分派，否则将错过截止期
- 与最早截止期优先相比它的优先级更加精确
- 最小裕度优先可调度的充分条件与最早截止期优先相同

提纲

3.1 中断与中断系统

3.2 处理器调度

3.3 调度级别与多级调度

3.4 实时调度

3.5 多处理器调度

3.6 系统举例

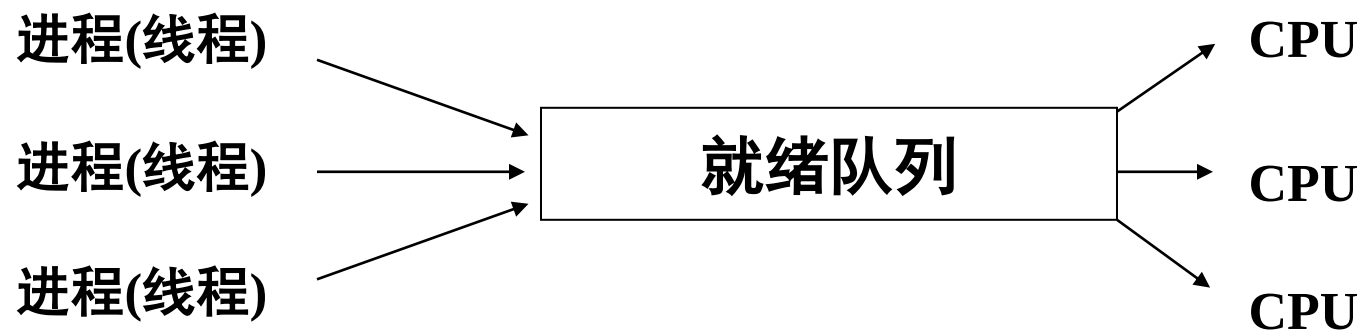
3.5 多处理器调度

- 问题：
 - M processes (threads)
 - N processors
- SMP: symmetric multi-processors
 - all processors are identical (homogeneous)
- 目标: load-sharing
 - separate ready queue for each processor
 - not really balanced
 - common ready queue Q for all processors
 - each processor accesses Q on its own

3.5.1 自调度(self scheduling)

- 均衡调度 (balanced scheduling)
- 一个就绪队列(多处理机访问互斥)
 - 优点
 - 不需要专门的处理机从事任务分派工作
 - 任务分配均衡
 - 缺点
 - 当CPU较多时, 就绪队列成为瓶颈
 - 线程两次调度可能处于不同处理机
 - 不能保证合作线程同时调度

自调度(self scheduling)



自调度(self scheduling)

- 例子：
 - Mach, 改进的自调度
 - 全局队列: 系统一个
 - 局部队列: 每个CPU一个
 - 调度时
 - 首先考虑局部队列
 - 然后考虑全局队列

3.5.2 组调度(gang scheduling)

- 将一组相关(合作)的线程同时分派到多个处理机上运行
 - 避免合作线程之间的相互等待
 - 降低开销, 提高运行效率
- 例子:
 - C_m^*
 - Task force (一组相关的计算)

提纲

3.1 中断与中断系统

3.2 处理器调度

3.3 调度级别与多级调度

3.4 实时调度

3.5 多处理器调度

3.6 系统举例

3.6 系统举例

- Linux进程调度
- Windows线程调度
- UNIX进程调度

3.6.1 Linux 进程调度

■ Goodness-based scheduling

■ priority

- 0-40, (缺省值20), 可通过nice系统调用调整, nice(value)中value的取值范围为(-20, 20)之间, 取 $\text{priority} = 20 - \text{value}$

■ counter

- 进程尚可运行的剩余时间, 单位是时钟滴答
 - 例: 时钟中断周期tick为10ms。若 $\text{counter} = 60$, 则能连续运行600ms
- 较高优先级的进程一般counter较大
- 一般把counter看作动态优先级
- Counter的初值与priority有关
 - 普通进程创建时counter的初值为priority的值
- Counter的改变
 - 对于运行进程, 每个时钟中断当前进程counter减1, 直到为0被阻塞
 - 当所有就绪进程的counter配额下降到0时, 重新计算所有进程(包括等待进程)的counter值
 - $\text{counter} = (\text{counter} / 2) + \text{priority}$

Linux 进程调度

- goodness
 - if (Real-time)
 $\text{goodness} = 1000 + \text{priority}$
 - if (Timesharing && counter=0)
 $\text{goodness} = 0$
 - if (Timesharing && counter>0)
 $\text{goodness} = \text{counter} + \text{priority}$

Linux 进程调度

- 调度发生时刻
 - 运行进程的counter减至0
 - 运行进程执行系统调用exit
 - 运行进程因等待I/O、信号灯而等待
 - 原来具有高goodness的进程被唤醒
- 调度效果
 - 实时进程优先于分时进程
 - 交互进程和I/O进程优先于CPU进程

Linux 对称多处理

- Linux 2.0 是支持对称多处理硬件的第一个Linux核心
 - 进程或线程可以同时运行在多个处理机上
 - 为保持核心非剥夺同步要求，SMP通过一个唯一的核**心自旋锁 (spin-lock)**来保证任何时刻最多只有一个处理器执行核心代码
 - 后来版本支持真正意义上的SMP：**将一个自旋锁分解为若干个相互独立的自旋锁，分别用于保护核心代码不相交的子集**

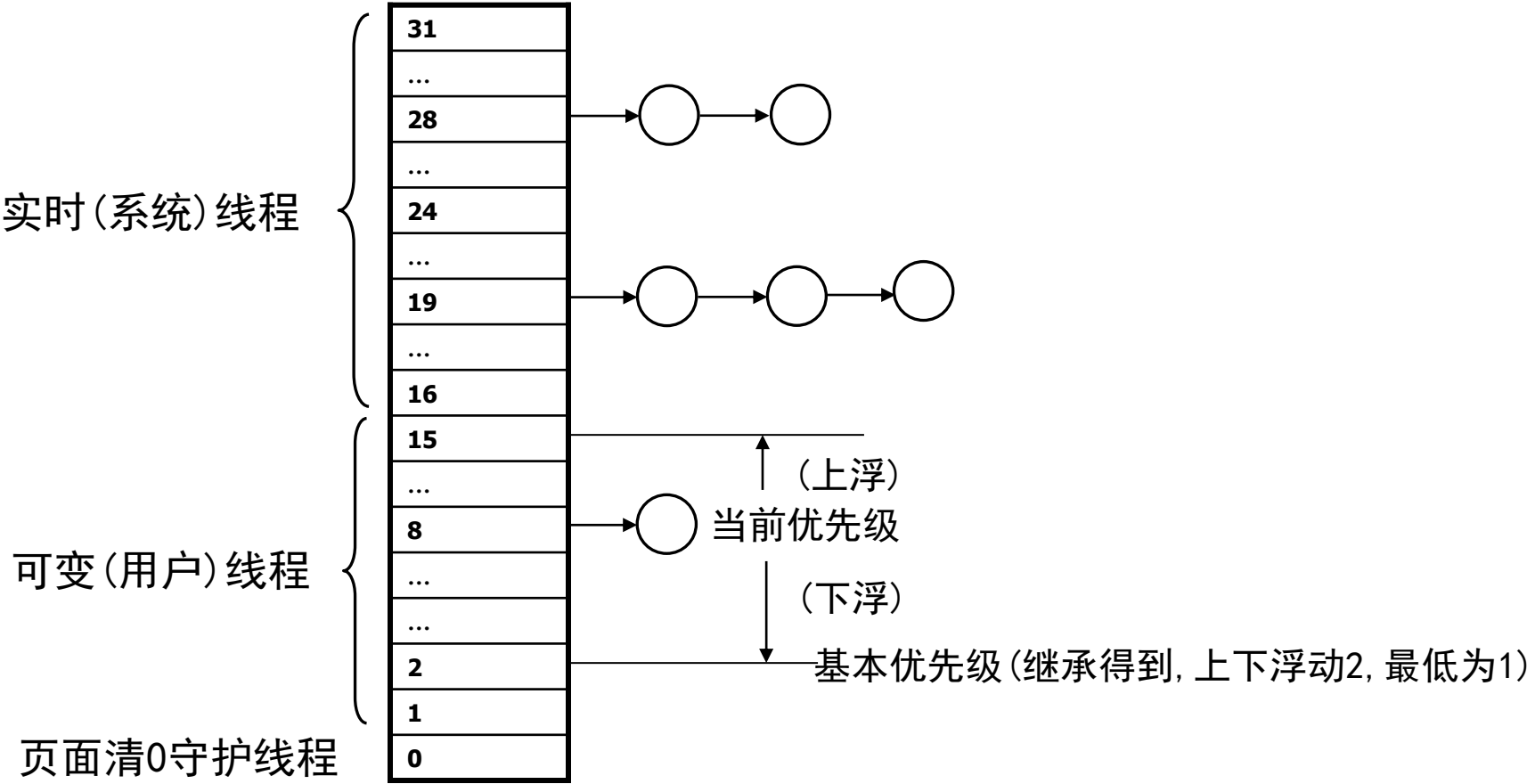
3.6.2 Windows 线程调度

- 主要特征
 - 以线程作为调度的主要单位
 - 同时具有real time + foreground + background特征的操作系统
 - real time: no deadline scheduling
 - foreground: GUI window
 - background: non-interactive
 - Preemptive + Dynamic Priority + RR + Feed Back
 - Symmetric Multi-Processor (SMP) support

优先级别

- 16个实时优先级（16-31）
 - 一些内核线程
 - 应用程序提升为实时优先级需要有权限
 - 不是真正意义上的实时调度
- 15个可变线程优先级（1-15）
 - 基本优先级
 - 线程基本优先级继承进程基本优先级，可上下浮动2
 - 如：进程基本优先级4，其线程基本优先级2—6，
 - 当前优先级
 - 在基本优先级与15之间浮动
 - 可动态提升
 - 运行完一个quantum之后自动下降，不低于基本优先级
- 1个页面清零守护线程优先级（0）

Windows优先级



优先级提升

- 优先级提升
 - IO操作完成
 - 事件等待结束
 - 前台进程中的线程完成一个等待操作
 - 由于窗口活动而唤醒GUI线程
 - 就绪超过一定时限，未获得处理机
- 用户线程优先级提升不会超过15

抢占CPU

- 抢先情形
 - 被唤醒线程优先级高于运行线程优先级
 - 某就绪线程的优先级动态变化
- 被抢先线程
 - 回到相应就绪队列
 - 时间配额
 - 实时线程：重新分配完整时间配额
 - 其它线程：保持剩余配额

时间配额(quantum)

- 配额长度：6(专业版)—36(服务器版)，服务器版中取较大时间配额是为保证应用程序有足够时间完成用户请求，并回到等待态
- 每次时钟中断(15ms发生一次)，系统将运行线程的时间配额减3。专业版一个线程的默认运行时间为2次时钟中断，服务器版中默认运行时间为12次时钟中断
- 配额用完后进入就绪队列，优先级下降

SMP上的线程调度

- 一个线程，尽量在同一个处理器上运行
- 线程与CPU的亲合关系
 - 每个进程有一个处理器亲合掩码，缺省为所有处理器的集合
 - 线程继承其进程的亲合掩码
 - 亲合掩码可以修改
 - `SetProcessAffinityMask`
 - `SetThreadAffinityMask`

SMP上的线程调度

- 线程的理想处理器（Ideal processor）
 - 首选处理器
 - 第二处理器
- 理想处理器确定
 - 线程创建时随机确定
 - 分散各个线程与处理机对应关系
 - 线程可修改SetThreadIdealProcessor

就绪线程对处理器的选择

- 有空闲处理器
 - 首选处理器
 - 第二处理器
 - 当前执行处理器（正执行调度代码）
 - 由高到低顺序找空闲的处理器
- 无空闲处理器，考虑抢先
 - 首选处理器
 - 第二处理器
 - 可运行编号最大处理器
- 不能抢先，进入相应的就绪队列

处理器对就绪线程的选择

- 空闲处理器调度
 - 线程上次在此CPU上运行（二级缓冲利用）
 - 线程的理想处理器是该CPU
 - 处于就绪状态时间超过2个quantum
 - 优先级别大于等于24

3.6.3 UNIX系统调度

- 例子UNIX: preemptive+dynamic priority (可抢占CPU 动态优先数)
 - 计算公式: $p_pri = \min\{127, \text{USER} + p_cpu/16 + p_nice\}$
 - $\text{USER}=100$
 - p_cpu
 - 运行进程每20ms加1 (优先级降低)
 - 其它进程每1200ms减10 (优先级提高)
 - p_nice : 可以通过系统调用命令 `nice(...)` 修改的量: 规定用户进程0~20之间 (低), 系统进程-20~+20之间 (高)
 - 调度时取 p_pri 最小的

作业 #1

1. 由核心返回目态程序时，进程的PSW和PC为何必须用一条机器指令同时恢复？

2. 对如下三个实时任务：

$T1=100, C1=50;$

$T2=200, C2=30;$

$T3=500, C3=100.$

采用EDF算法和RMS算法是否可调度？如是画出对应的Gantt图，否则说明原因。

3. 若程序PA和PB单独执行时间分别为TA和TB， $TA=1h$ ， $TB=1.5h$ ，其中处理器工作时间分别为18min和27min。如果采用多道程序设计方法，让PA、PB同时工作，假定处理器利用率达到50%，另加15min系统开销，问系统效率能提高百分十几？
4. 假设一个计算机系统具有如下特征：处理一次中断，平均耗时1ms。一次进程调度，平均需要2ms。将CPU分配给选中的进程，又平均需要1ms。再假设其定时器芯片每秒产生100次中断。请回答：操作系统将百分之几的CPU时间用于时钟中断处理？如果操作系统采用循环轮转法调度，10个时钟中断为一个时间片。那么，操作系统将百分之几的CPU时间用于进程调度？（包括调度、分配CPU和引起调度的时钟中断处理时间）